



中国研究生创新实践系列大赛
“华为杯”第二十一届中国研究生
数学建模竞赛

学 校

武汉大学

参赛队号

24104860143

队员姓名

1.

赵龙

2.

付丙臣

3.

刘泽

中国研究生创新实践系列大赛
“华为杯”第二十一届中国研究生
数学建模竞赛

题 目： X 射线脉冲星光子到达时间建模

摘 要：

脉冲星由于自转的连续性和稳定性，其被认为是宇宙中最精确的时钟。脉冲星可以提供独立、稳定的空间参考基准和时间基准，为空间飞行提供导航信标。基于上述背景，本文将“X 射线脉冲星光子到达时间的精确建模”划分为四个子问题，首先，从理想的二体运动模型切入，充分考虑时空参考系统转换及对光子到达时间精确估计的影响因素，构建精确反映脉冲光子辐射特性的函数模型，并采取涵盖“问题分析、模型建立、模型求解与结果评估”四个环节的完备研究流程。

针对问题一，本文采用理想的二体运动模型，严格推导了开普勒轨道根数与卫星位置、速度之间的函数关系，最终求解得到卫星的位置为 $X=1274.9134151\text{km}$, $Y=-1848.8519650\text{km}$, $Z=6507.2626553\text{km}$, 速度为 $V_x=-6.2107951\text{km/s}$, $V_y=3.7461273\text{km/s}$, $V_z=2.2763309\text{km/s}$ 。为了验证轨道参数的一致性以及计算结果的准确性，并通过反算轨道根数及利用 θ 角的时变特性预报一个完整周期的卫星轨道进一步验证结果。结果表明反算得到的轨道根数数值与题目所提供的轨道根数数值完全相等并且 θ 改变时，由于不考虑摄动力以及第三体引力等因素的影响，卫星的轨道变化得十分平稳和光滑，有力地证明了轨道参数一致性和计算结果的准确性。

针对问题二，本文充分考虑了卫星处与太阳系质心的坐标系与时间系统转换并顾及脉冲星自行对时延的影响，假设 X 射线光子信号为平行光且忽略太阳系天体的自转和扁率，建立了脉冲星光子到达卫星与太阳系质心的传播路径时间差模型。首先将地球时 TT 时间系统转换为太阳系质心力学时 TDB 时间系统，后将坐标系由地心天球坐标系转至太阳质心天球坐标系，使用脉冲星自行参数修正赤经和赤纬，最终精确求解得到时间差为 $-277.92366023979565\text{s}$ 。

针对问题三，本文考虑了卫星与其他天体的相对位置关系，并对几何传播时延、Shapiro 时延、引力红移时延和狭义相对论的动钟变慢效应的产生原因进行了精确的时延转换模型建立与求解，最终获得了精确的脉冲光子传播时延。各项时延如下：几何传播时延 $\Delta t_{\text{geo}} = -473.88401304073534\text{s}$, Shapiro 时延 $\Delta t_{\text{Shapiro}} = -6.6254740275614095 \times 10^{-6}\text{s}$, 引力红移时

延 $\Delta t_{\text{redshift}} = -9.979327322105222 \times 10^{-9} s$, 狭义相对论时延 $\Delta t_{\text{rel}} = -7.027050984279227 \times 10^{-10} s$, 最终计算的总时延结果为: $\Delta t_{\text{total}} = -473.8840196768914 s$

针对问题四, 本文利用已知条件归一化后的标准脉冲轮廓曲线数据和 Crab 脉冲星的自转参数及流量密度数据建立了脉冲光子仿真时间序列模型, 成功实现 Crab 脉冲星光子到达的时间序列仿真, 并仿真的脉冲轮廓结果与标准脉冲轮廓高度一致, 两者之间相关性最高达到了 **0.99351**, 这证明了本文所使用的模型具有很高的仿真精度。此外, 本文利用 χ^2 拟合优度对使用的脉冲模型进行了检验, 检验结果表明本文模拟的脉冲光子时间序列很好地服从泊松分布, 确保模型具有良好的适用性和稳健性。另外, 考虑到实际脉冲光子的流量有时是比较少的, 不满足两个光子到达时间间隔趋近于 0 的条件, 因此本文提出了一种逆变换采样法来进行建模, 结果表明基于逆变换采样法得到的仿真脉冲轮廓与标准脉冲轮廓的一致性更好, 可以更好地展现脉冲星的辐射特性。

关键词: 脉冲星 X 射线光子 精确时延转换 光子序列仿真 非条件泊松分布

目录

1	问题重述	5
1.1	问题背景	5
1.2	问题提出	5
1.3	总体研究路线	6
2	模型假设和符号说明	7
2.1	模型假设	7
2.2	符号说明	7
3	问题一的建模与求解	8
3.1	问题分析	8
3.2	模型建立	9
3.2.1	由轨道根数计算位置、速度	15
3.2.2	由位置、速度反算轨道根数	16
3.3	模型求解与结果分析	17
4	问题二的建模与求解	21
4.1	问题分析	21
4.2	模型建立	22
4.3	模型求解与结果分析	25
5	问题三的建模与求解	27
5.1	问题分析	27
5.2	模型建立	28
5.3	模型求解与结果分析	30
6	问题四的建模与求解	33
6.1	问题分析	33
6.2	模型建立	34
6.2.1	脉冲星光子序列仿真	34
6.2.2	脉冲星轮廓构建及检验	37
6.2.3	仿真精度提高	38

6.3 模型求解与结果分析	40
7 模型评价	49
参考文献	50
附录 A Python 源程序	51
A.1 第 1 问程序	51
A.2 第 2 问程序	55
A.3 第 3 问程序	57
A.4 第 4 问 1、2 小问程序	61
A.5 第 4 问 1、2 小问检验程序	68
A.6 第 4 问 3 小问程序	72

1 问题重述

1.1 问题背景

脉冲星导航作为未来深空探测任务中的重要技术之一，能够为卫星提供高精度的自导航能力。与传统的地基导航相比，脉冲星导航利用稳定的周期性辐射信号进行导航，尤其是在深空探测任务中，地面信号难以实时提供精确定位信息，脉冲星导航成为解决这一问题的关键技术。脉冲星，特别是 X 射线脉冲星，因其具有极其稳定的自转周期和强烈的电磁辐射，成为可用于深空导航的理想信号源。

然而，利用脉冲星实现精确的导航和定位依赖于对脉冲星光子到达时间的准确建模。在卫星高速运动、距离较远的背景下，脉冲星光子在传播过程中会受到一系列影响因素的干扰，例如几何传播时延、引力场效应、太阳系天体的运动等。这些因素共同作用，使得脉冲星光子到达时间的精确计算变得极为复杂。

因此，如何在不同参考系下建立脉冲星光子的到达时间模型，计算其传播路径的时延，校正传播过程中的时延误差，成为脉冲星导航领域的重要研究课题。通过对这些问题的深入研究，不仅能够提高脉冲星导航的精度，也能为未来的深空探测任务提供更为可靠的自导航手段。

1.2 问题提出

脉冲星导航作为未来深空航天任务中的关键技术，其精度和可靠性直接取决于对脉冲星光子到达时间的准确建模。在此背景下，本研究旨在探讨脉冲星光子到达时间的建模方法，并解决与此相关的若干关键问题。具体而言，本研究将围绕以下几个核心问题展开：

问题一：利用给定的轨道根数（如偏心率、半长轴、轨道倾角等）计算卫星在地心天球参考系（GCRS）中的三维位置和速度。同时，验证计算的轨道参数与结果的一致性，确保计算结果的精度。

问题二：假设脉冲星发射的 X 射线光子为平行光，忽略太阳系天体的自转和扁率，基于问题一中确定的卫星位置坐标，并在得到其在太阳系质心坐标系中位置基础上，计算脉冲星光子到达卫星与太阳系质心的传播路径差，将其转化为传播时延。此模型将为进一步计算精确时延提供基础。

问题三：在几何传播时延基础上，综合考虑 Shapiro 时延、引力红移、狭义相对论效应等关键时延因素，以及脉冲星自行的影响，建立精确的脉冲星光子到达卫星的时延模型。该模型旨在消除影响脉冲星导航精度的各类时延因素。

问题四：构建 X 射线脉冲星光子序列的数学模型，并进行仿真实验，以分析脉冲星信号的辐射过程。通过仿真 Crab 脉冲星光子序列，结合其自转参数，折叠出脉冲轮廓，并在此基础上研究如何提升仿真精度，展示更真实的脉冲星辐射特性。

1.3 总体研究路线

针对上述问题，本研究总体路线如图1.1所示，说明了各问题间存在的关联。

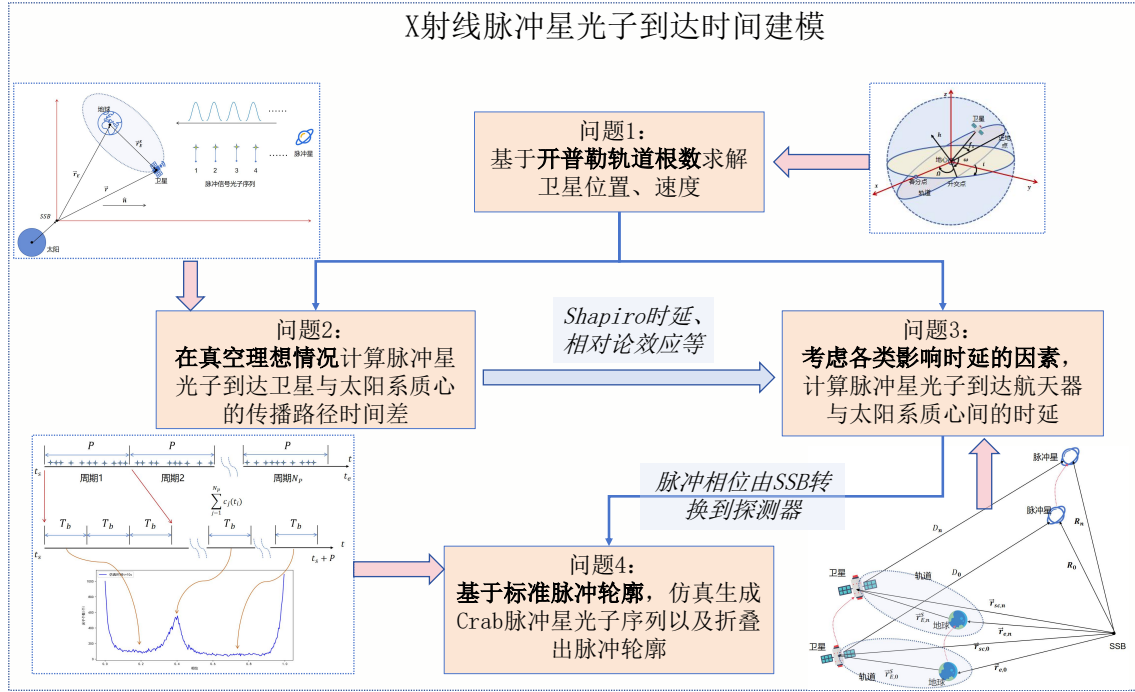


图 1.1 总体研究路线

问题一所计算的卫星位置和速度，需应用到问题二、三以计算各类时延因素，问题三在问题二的基础上，考虑除几何传播时延外的 Shapiro 时延、引力红移时延和相对论效应并对这些因素进行建模，以建立精确转换时延模型，问题四则需基于所建立的精确实验模型将在 SSB 处模拟的光子到达时间序列转至探测器处，并进一步折叠出脉冲轮廓，与标准轮廓对比，评估模型的性能。

2 模型假设和符号说明

2.1 模型假设

1. 假设相邻脉冲光子到达时间间隔较小，时间序列符合非齐次泊松分布；
2. 假设 X 射线光子信号为平行光；
3. 忽略太阳系天体自转和扁率；
4. 太阳是主要的引力源，为了便于讨论及建模，在计算只考虑由光子经过太阳时的引力场所产生 Shapiro 时延和引力红移时延；
5. 假设脉冲星的自转频率在短时间内是稳定的；
6. 二体运动模型不考虑摄动改正及第三体影响；

2.2 符号说明

符号	意义
ϕ	脉冲相位
λ	脉冲星的流量密度函数
h	归一化的脉冲轮廓函数
$h(\phi)$	Crab 脉冲星的标准脉冲轮廓曲线
λ_b	背景光子流量密度
λ_S	Crab 脉冲星光子流量密度
Λ	积分速率函数
TOA	到达时间
POA	到达相位
R_{sp}	Pearson 系数
R	[0,1] 上均匀分布的随机变量
N	子相位间隔数
NHPP	非齐次泊松分布
χ^2	皮尔逊统计量
τ_i	参数为 1 的齐次泊松过程的 TOA 序列

3 问题一的建模与求解

针对问题一，本文采取的模型建立与求解框架如图3.1所示。问题一的已知条件为六个开普勒轨道根数，需要根据这些参数求解卫星在地心参考坐标系下 GCRS 的位置和速度结果。由于 GCRS 坐标系本身就为惯性系，因此在根据开普勒运动规律解算出结果后并不需要额外进行坐标系的转换。

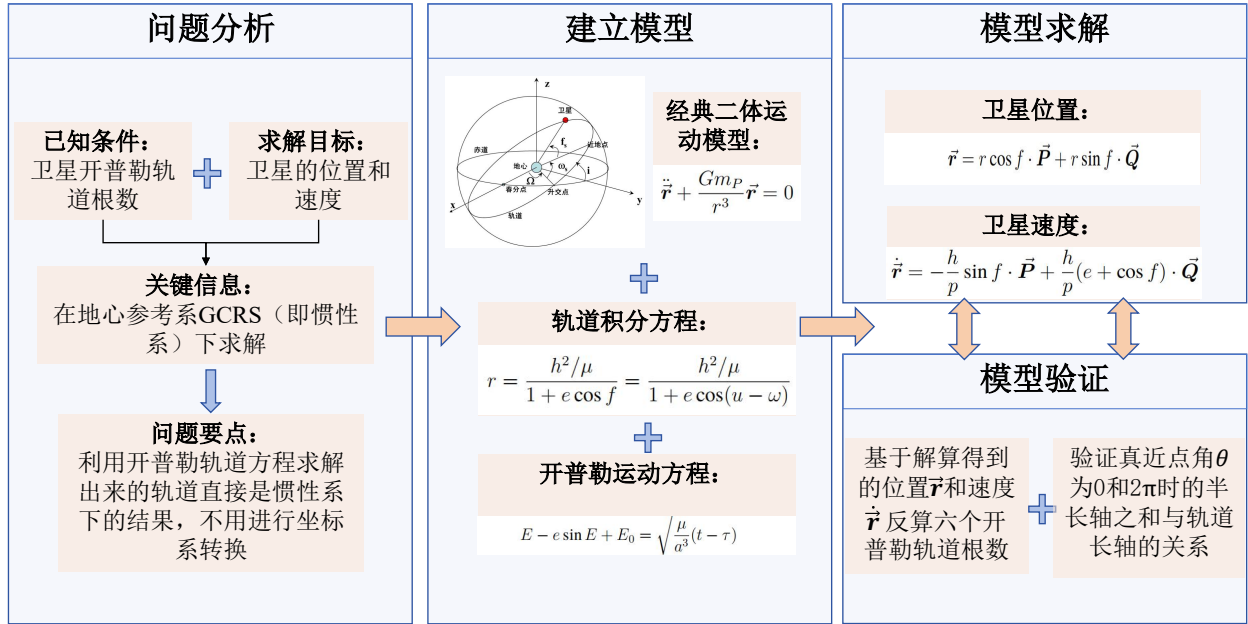


图 3.1 问题一整体框架示意图

基于上述对问题一的分析，可以根据经典二体运动模型建立卫星与地球的轨道积分方程和开普勒运动方程，并利用已知条件（六个卫星轨道开普勒根数）进行模型求解获得卫星的位置和速度。在求解出结果后需要进行相应验证，本研究采用基于求解出的卫星位置和速度反算开普勒轨道根数的正向与逆向相结合的验证思路，同时利用真近点角 θ 为0和 2π 时半长轴之和与轨道长轴间的关系两种方法对模型求解的结果进行验证，以保证结果的准确性和可靠性。

3.1 问题分析

本题目要求根据卫星的轨道根数计算卫星在特定时刻的三维位置和速度，卫星的开普勒轨道根数在空间上的关系如图3.2所示，具体包括以下六个独立参数：

- **偏心率 e** ：描述轨道椭圆形状的参数。
- **角动量 h** ：表示卫星的轨道角动量。
- **轨道倾角 i** ：卫星轨道平面相对于赤道平面的倾斜角度。
- **真近点角 θ** ：卫星在轨道上的位置角。

- 升交点赤经 Ω : 卫星通过赤道升交点时的赤经。
- 近地点幅角 ω : 近地点与升交点之间的角度。

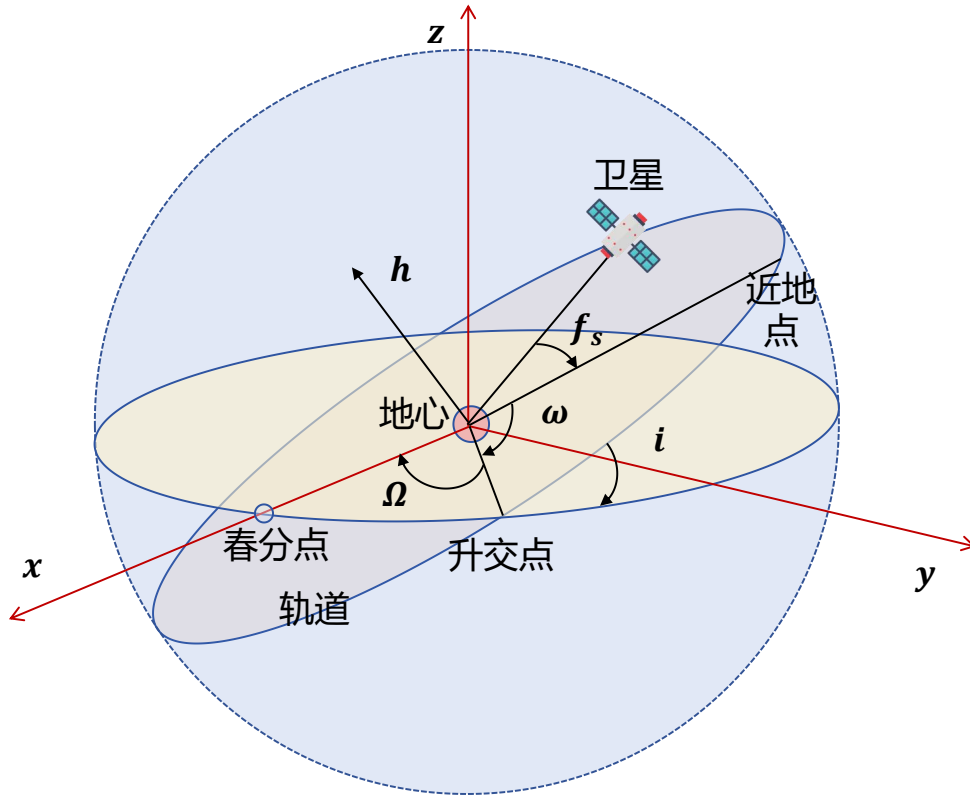


图 3.2 开普勒轨道根数

通过这六个参数，就可以唯一确定卫星在轨道平面中的位置，并通过坐标变换将卫星在轨道平面中的位置和速度变换到空间三维直角坐标系，最终获得卫星在 GCRS 系中的位置和速度。

3.2 模型建立

由于问题一不需要考虑除地球之外的第三体引力影响，同时忽略掉大气、太阳光压等摄动改正的影响，因此问题一可以简化为一个经典的二体问题。在二体问题中，通常可以考虑有一个中心天体，且中心天体的质量要远大于另一个天体，即若 $m_P \gg m_Q$ ，则天体 Q 不会影响天体 P 的运动（实际上，影响可忽略不计）。在此假定下，以天体 P 为坐标系原点，二体问题（ Q 绕 P 的轨道运动）运动方程可以表示为

$$\ddot{\vec{r}} + \frac{Gm_P}{r^3} \vec{r} = 0 \quad (3.1)$$

其中， G 为万有引力常数， m_P 为中心天体的质量， $\vec{r}$ 为天体 Q 相对于 P 的位置向量。令 $\mu = Gm_P$ ，则上式可以写为：

$$\ddot{\vec{r}} + \frac{\mu}{r^3} \vec{r} = 0 \quad (3.2)$$

二体问题主要分别从三个重要的积分（面积积分、轨道积分和活力积分）来解决，下面首先介绍面积积分。定义行星位置矢量 $\vec{r}$ 和速度矢量 $\dot{\vec{r}}$ 的叉乘为 $\vec{h}$ ，即：

$$\vec{r} \times \dot{\vec{r}} = \vec{h} \quad (3.3)$$

其中， $\vec{h} = \begin{bmatrix} h_x \\ h_y \\ h_z \end{bmatrix}$ ，则根据向量运算以及导数的运算法则，有

$$\frac{d\vec{h}}{dt} = \frac{d}{dt}(\vec{r} \times \dot{\vec{r}}) = \dot{\vec{r}} \times \dot{\vec{r}} + \vec{r} \times \ddot{\vec{r}} = \vec{r} \times \ddot{\vec{r}} \quad (3.4)$$

将式 (3.2) 的两边同时叉乘位置向量 $\vec{r}$ 可得

$$\vec{r} \times \ddot{\vec{r}} + \frac{\mu}{r^3}(\vec{r} \times \vec{r}) = \vec{r} \times \ddot{\vec{r}} = 0 \quad (3.5)$$

所以

$$\frac{d\vec{h}}{dt} = 0 \quad (3.6)$$

通过上式可知，向量 $\vec{h}$ 为与时间无关的常值向量，其对时间的导数为零，进而可得出以下结论：

- 二体问题的动量矩是守恒的。
- $\vec{h}$ 表示单位质量的动量矩。行星绕太阳公转的角动量守恒。
- $\vec{h}$ 垂直于轨道运动平面，因此轨道面在惯性坐标系中是固定的。 $\vec{h}$ 代表轨道面的法向。

可以用轨道倾角 i 和升交点赤经 Ω 表示

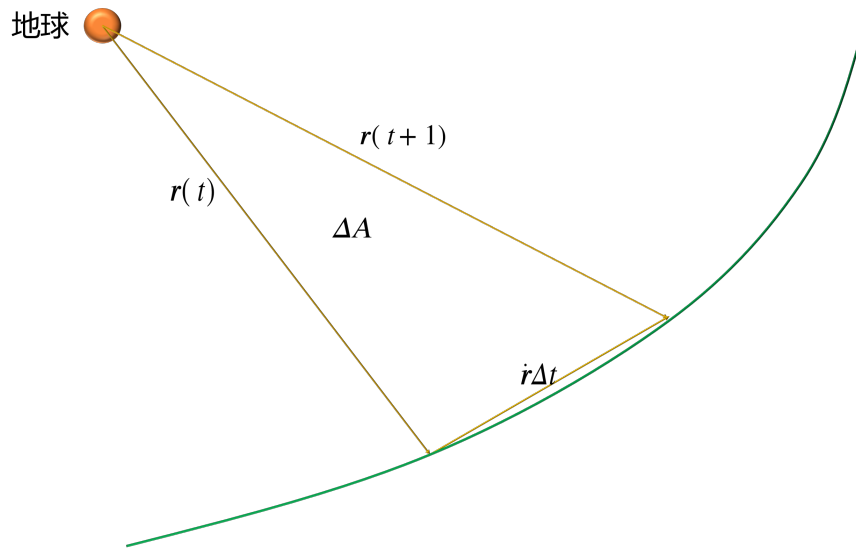


图 3.3 卫星扫过面积图

如图3.3所示， Δt 时间内扫过的面积为：

$$\Delta A = \frac{1}{2} |\vec{r} \times \dot{\vec{r}} \Delta t| \quad (3.7)$$

又由式 (3.3) 可得

$$\Delta A = \frac{1}{2} |\vec{r} \times \dot{\vec{r}} \Delta t| = \frac{1}{2} |\vec{h}| \Delta t \quad (3.8)$$

所以

$$2 \frac{dA}{dt} = |\vec{h}| = h \quad (3.9)$$

可以定义 $\frac{dA}{dt}$ 为卫星轨道的面积速度，则 h 为面积速度的两倍。图3.4为卫星轨道的示意图，则有 $h = rv \sin \gamma = rv \cos \theta$ ，其中 θ 为水平航迹角，又称为速度倾角。

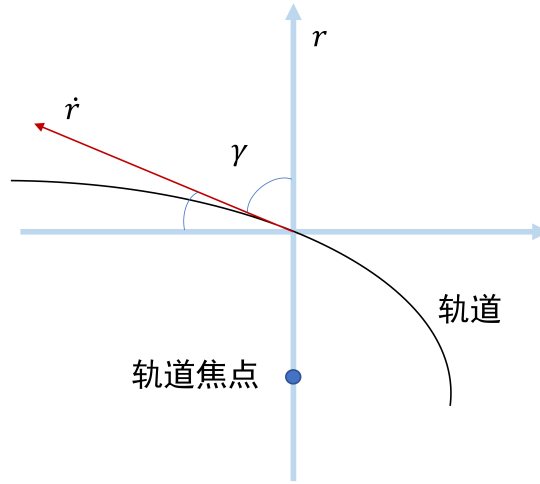


图 3.4 卫星轨道示意图

假设卫星在 dt 时间内转动 du 大小的角度，则有

$$dA = \frac{1}{2} r \cdot r du \quad (3.10)$$

则

$$\frac{dA}{dt} = \frac{1}{2} r^2 \frac{du}{dt} = \frac{1}{2} r^2 \dot{u} \quad (3.11)$$

又根据式 (3.9) 可得

$$h = r^2 \dot{u} \quad (3.12)$$

将式 (3.2) 与 $\vec{h}$ 叉乘可得

$$\begin{aligned} \ddot{\vec{r}} \times \vec{h} &= -\frac{\mu}{r^3} (\vec{r} \times \vec{h}) \\ &= -\frac{\mu}{r^3} (\vec{r} \times \vec{r} \times \dot{\vec{r}}) \end{aligned} \quad (3.13)$$

因为三个向量的叉乘有以下关系：

$$\vec{a} \times \vec{b} \times \vec{c} = (\vec{a} \cdot \vec{c}) \cdot \vec{b} - (\vec{a} \cdot \vec{b}) \cdot \vec{c} \quad (3.14)$$

则有：

$$\begin{aligned} \ddot{\vec{r}} \times \vec{h} &= -\frac{\mu}{r^3} [(\vec{r} \cdot \dot{\vec{r}}) \cdot \vec{r} - (\vec{r} \cdot \vec{r}) \dot{\vec{r}}] \\ &= \frac{\mu}{r^3} [r^2 \dot{\vec{r}} - (r\dot{r})\vec{r}] \\ &= \mu \frac{d}{dt} \left(\frac{\vec{r}}{r} \right) \end{aligned} \quad (3.15)$$

对上式两边同时取积分，则有

$$\dot{\vec{r}} \times \vec{h} = \frac{\mu}{r} \vec{r} + \mu \vec{e} \quad (3.16)$$

上式中 $\vec{e}$ 为积分常矢量。

又因为向量混合积有以下性质：

$$(\vec{a} \times \vec{b}) \cdot \vec{c} = (\vec{b} \times \vec{c}) \cdot \vec{a} \quad (3.17)$$

则

$$(\dot{\vec{r}} \times \vec{h}) \cdot \vec{h} = \dot{\vec{r}} \cdot (\vec{h} \times \vec{h}) = 0 \quad (3.18)$$

并且根据 $\vec{h}$ 的定义可知 $\vec{h}$ 垂直于位置向量 $\vec{r}$ ，则有

$$\begin{aligned} (\dot{\vec{r}} \times \vec{h}) \cdot \vec{h} &= \frac{\mu}{r} \vec{r} \cdot \vec{h} + \mu \vec{e} \cdot \vec{h} \\ &= \mu \vec{e} \cdot \vec{h} = 0 \end{aligned} \quad (3.19)$$

故

$$\vec{e} \cdot \vec{h} = 0 \quad (3.20)$$

从上式可以看出， $\vec{e}$ 位于轨道平面内；积分常数中只有两个独立的积分常数， e 和 ω ， $\vec{e}$ 向量是在轨道平面位于 X 轴上模长为 e （即偏心率）的向量，其在轨道坐标系下的坐标为

$$\vec{e} = \begin{bmatrix} e \\ 0 \\ 0 \end{bmatrix}, \text{ 在惯性坐标系下的坐标为 } \vec{e} = \begin{bmatrix} e_x \\ e_y \\ e_z \end{bmatrix}$$

对式 (3.16) 的两边同时点乘位置向量 $\vec{r}$ 可得出

$$\begin{cases} (\dot{\vec{r}} \times \vec{h}) \cdot \vec{r} = \vec{h} \cdot (\vec{r} \times \dot{\vec{r}}) = \vec{h} \cdot \vec{h} = h^2 \\ \frac{\mu}{r} (\vec{r} + r\vec{e}) \cdot \vec{r} = \mu r + \mu r e \cos f \end{cases} \quad (3.21)$$

上式中的 f 即为位置向量与轨道椭圆的长半轴（对应近地点）的夹角，即为真近点角，其只与时间有关。化简上式，可得

$$h^2 = \mu r (1 + e \cos f) \quad (3.22)$$

将位置向量的模移到左边，有

$$r = \frac{h^2/\mu}{1 + e \cos f} = \frac{h^2/\mu}{1 + e \cos(u - \omega)} \quad (3.23)$$

上式中， u 为 $\vec{r}$ 和轨道平面的 OX' 轴之间的夹角，即升交距角； ω 为 $\vec{e}$ 和 OX' 之间的夹角，即轨道椭圆长半轴（对应近地点）与轨道平面坐标系的 OX' ，也就是所谓的近地点角距。

椭圆的半通径公式为

$$p = \frac{b^2}{a} = \frac{a^2 - c^2}{a} = a(1 - \frac{c^2}{a^2}) = a(1 - e^2) \quad (3.24)$$

并且半通径有下面关系

$$p = \frac{h^2}{\mu} = a(1 - e^2) \quad (3.25)$$

因此轨道积分有以下几种表达形式：

$$\begin{aligned} r &= \frac{h^2/\mu}{1 + e \cos f} \\ &= \frac{p}{1 + e \cos f} \\ &= \frac{a(1 - e^2)}{1 + e \cos f} \end{aligned} \quad (3.26)$$

由于 $\dot{f} = \dot{u}$ ，所以根据式 (3.12) 可得

$$\dot{f} = \frac{h}{r^2} \quad (3.27)$$

又因为

$$r = \frac{p}{1 + e \cos f} \quad (3.28)$$

则

$$\dot{f} = \frac{h(1 + e \cos f)^2}{p^2} \quad (3.29)$$

又因为

$$p = \frac{h^2}{\mu} \quad (3.30)$$

将上式代入式 (3.29) 可得

$$\dot{f} = \sqrt{\frac{\mu}{p^3}} (1 + e \cos f)^2 \quad (3.31)$$

写成微分形式为：

$$dt = \sqrt{\frac{p^3}{\mu}} \frac{1}{(1 + e \cos f)^2} df \quad (3.32)$$

对上式左右两边取定积分得到

$$t - t_0 = \sqrt{\frac{p^3}{\mu}} \int_{f_0}^f \frac{df}{(1 + e \cos f)^2} \quad (3.33)$$

从上式中可以看出卫星在轨道平面中的真近点角 f 只与其运动时间 t 有关，由于直接在椭圆轨道上计算 f 不方便，因此以椭圆轨道的中心为圆心，半长轴为半径作一个圆。同时定义 E 为辅助圆对应角度，称之为偏近点角。具体定义如下：

$$\begin{aligned} \hat{x} &= r \cos f = a(\cos E - e) \\ \hat{y} &= r \sin f = a\sqrt{1 - e^2} \sin E \end{aligned} \quad (3.34)$$

或者可等价为：

$$r = a(1 - e \cos E) \quad (3.35)$$

其导数为

$$\dot{r} = ae \sin E \cdot \dot{E} \quad (3.36)$$

又因为

$$\dot{r} = \sqrt{\frac{\mu}{p}} e \sin f \quad (3.37)$$

所以

$$\dot{E} = \sqrt{\frac{\mu}{a^3(1 - e^2)}} \cdot \frac{\sin f}{\sin E} \quad (3.38)$$

并将式 (3.34) 代入上式，化简可得

$$(1 - e \cos E) \frac{dE}{dt} = \sqrt{\frac{\mu}{a^3}}$$

即

$$(1 - e \cos E) dE = \sqrt{\frac{\mu}{a^3}} dt \quad (3.39)$$

对上式左右两边取积分得到

$$E - e \sin E + E_0 = \sqrt{\frac{\mu}{a^3}} (t - \tau) \quad (3.40)$$

上式中， τ 为卫星过近地点的时刻。则式 (3.40) 可化为

$$E - e \sin E = M \quad (3.41)$$

在实际计算时，求解 E 需要进行循环迭代计算，也可利用牛顿迭代法进行计算：

$$E_{i+1} = E_i - \frac{E_i - e \sin E_i - M}{1 - e \cos E_i} \quad (3.42)$$

当 $E = 0$ 时， $\tau = t$ ，此时卫星在近地点处。可以通过式 (3.34) 消去 r ，得

$$\tan f = \frac{\sqrt{1 - e^2} \sin E}{\cos E - e} \quad (3.43)$$

对上式求反正切函数，即可得到真近点角 f 。经过上述推导，至此完整得到了六个开普勒轨道根数。

3.2.1 由轨道根数计算位置、速度

根据轨道积分公式，在轨道坐标系下，某时刻的卫星坐标为

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} r \cos f \\ r \sin f \\ 0 \end{bmatrix} \quad (3.44)$$

根据坐标变换可得

$$\vec{r} = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = R_3(-\Omega)R_1(-i)R_3(-\omega) \begin{bmatrix} r \cos f \\ r \sin f \\ 0 \end{bmatrix} \quad (3.45)$$

通过上式即可计算得到位置矢量 $\vec{r}$ 。为了计算速度矢量 $\dot{\vec{r}}$ ，分别令

$$\vec{P} = R_3(-\Omega)R_1(-i)R_3(-\omega) \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} \cos \Omega \cos \omega - \sin \Omega \sin \omega \cos i \\ \sin \Omega \cos \omega + \cos \Omega \sin \omega \cos i \\ \sin \omega \sin i \end{bmatrix} \quad (3.46)$$

$$\vec{Q} = R_3(-\Omega)R_1(-i)R_3(-\omega) \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -\cos \Omega \sin \omega - \sin \Omega \cos \omega \cos i \\ -\sin \Omega \sin \omega + \cos \Omega \cos \omega \cos i \\ \cos \omega \sin i \end{bmatrix} \quad (3.47)$$

则式 (3.45) 可以写为：

$$\vec{r} = r \cos f \cdot \vec{P} + r \sin f \cdot \vec{Q} \quad (3.48)$$

对上式求导得到

$$\dot{\vec{r}} = (\cos f \cdot \vec{P} + \sin f \cdot \vec{Q})\dot{r} + (-r \sin f \cdot \vec{P} + r \cos f \cdot \vec{Q})\dot{f} \quad (3.49)$$

根据前面已经得到的

$$\begin{cases} \dot{f} = \frac{h}{r^2} \\ h^2 = \mu p \\ r = \frac{p}{1+e \cos f} \\ p = a(1-e^2) \\ \dot{r} = \frac{h}{p} e \sin f \end{cases} \quad (3.50)$$

将上式代入 (3.49) 中，最终可以得到

$$\dot{\vec{r}} = -\frac{h}{p} \sin f \cdot \vec{P} + \frac{h}{p} (e + \cos f) \cdot \vec{Q} \quad (3.51)$$

通过上式即可计算得到卫星运动的速度。因此，通过上述推导可以得到由开普勒轨道根数求解卫星位置的函数模型。

3.2.2 由位置、速度反算轨道根数

由于轨道根数和卫星位置与速度是一一对应的，因此利用给定的位置和速度，有且只有一组轨道根数与之相对应，因此，为了对轨道参数一致性和计算结果进行验证，采用对求解得到的卫星位置和速度反算其轨道根数，相应原理如下：

根据面积积分，卫星在惯性坐标系下的 $\vec{h}$:

$$\vec{r} \times \dot{\vec{r}} = \vec{h} = \begin{bmatrix} y\dot{z} - z\dot{y} \\ z\dot{x} - x\dot{z} \\ x\dot{y} - y\dot{x} \end{bmatrix} = \begin{bmatrix} h_x \\ h_y \\ h_z \end{bmatrix} \quad (3.52)$$

易知在轨道坐标系下:

$$\vec{h}' = \begin{bmatrix} 0 \\ 0 \\ h \end{bmatrix} \quad (3.53)$$

由坐标变换关系可得

$$\begin{aligned} \vec{h} &= \mathbf{R}_3(-\Omega) \mathbf{R}_1(-i) \mathbf{R}_3(-u) \vec{h}' \\ &= \begin{bmatrix} \cos \Omega \cos u - \sin \Omega \sin u \cos i & -\cos \Omega \sin u - \sin \Omega \cos u \cos i & \sin \Omega \sin i \\ \sin \Omega \cos u + \cos \Omega \sin u \cos i & -\sin \Omega \sin u + \cos \Omega \cos u \cos i & -\cos \Omega \sin i \\ \sin u \sin i & \cos u \sin i & \cos i \end{bmatrix} \vec{h}' \end{aligned} \quad (3.54)$$

将式 (3.53) 代入上式可得

$$\begin{bmatrix} h_x \\ h_y \\ h_z \end{bmatrix} = \mathbf{R}_3(-\Omega) \mathbf{R}_1(-i) \mathbf{R}_3(-u) \begin{bmatrix} 0 \\ 0 \\ h \end{bmatrix} = \begin{bmatrix} h \sin \Omega \sin i \\ -h \cos \Omega \sin i \\ h \cos i \end{bmatrix} \quad (3.55)$$

根据上式，可得轨道倾角 i 和升交点赤经 Ω 分别满足

$$\cos i = \frac{h_z}{h} \quad (3.56)$$

$$\tan \Omega = \frac{h_x}{h_y} \quad (3.57)$$

其中， $h = \sqrt{h_x^2 + h_y^2 + h_z^2}$ 。

通过式 (3.56)、(3.57) 即可求解得到轨道倾角 i 和升交点赤经 Ω 。面积速度可得半通径为：

$$p = \frac{h^2}{GM} \quad (3.58)$$

由活力公式可得半长轴为：

$$a = \left(\frac{2}{r} - \frac{v^2}{GM} \right)^{-1} \quad (3.59)$$

平运动角速度为：

$$n = \sqrt{\frac{GM}{a^3}} \quad (3.60)$$

对于椭圆轨道，半长轴 a 始终为正值，因此偏心率可表示为：

$$e = \sqrt{1 - \frac{p}{a}} \quad (3.61)$$

考虑式 (3.35) 和下面的恒等式：

$$\begin{aligned} r \cdot \dot{r} &= -a(\cos(E) - e) \cdot a \sin(E) \dot{E} \\ &+ a\sqrt{1 - e^2} \sin(E) \cdot a\sqrt{1 - e^2} \cos(E) \dot{E} \\ &= a^2 n e \sin(E) \end{aligned} \quad (3.62)$$

可求得偏近点角如下：

$$E = \arctan \left(\frac{\mathbf{r} \cdot \dot{\mathbf{r}} / (a^2 n)}{1 - r/a} \right) \quad (3.63)$$

根据开普勒方程，可通过偏近点角求解平近点角：

$$M(t) = E(t) - e \sin E(t) \quad (3.64)$$

为了求解近地点辐角 ω ，需先计算升交角距 u 如下：

$$\tan u = \frac{Z}{(Y \sin \Omega + X \cos \Omega) \sin i} \quad (3.65)$$

此外真近点角为：

$$f = \arctan \left(\frac{\sqrt{1 - e^2} \sin E}{\cos E - e} \right) \quad (3.66)$$

最终得近地点辐角为：

$$\omega = u - f \quad (3.67)$$

3.3 模型求解与结果分析

在给定卫星的轨道六要素后，首先计算长半轴和卫星的半通径，它们是描述轨道形状的两个基本量。长半轴定义了卫星的平均距离，半通径则表示轨道的形状。通过这些参数，可以进一步求解卫星的瞬时距离和角动量

接下来，通过定义的两个方向向量 $\mathbf{P}$ 和 $\mathbf{Q}$ ，可将卫星的轨道坐标系转换到地心惯性坐标系中。这个步骤涉及到将轨道平面投影到三维空间直角坐标系，最终得出卫星在 GCRS 系的位置和速度。求解的详细过程如下表所示：

算法 1 从轨道根数求位置和速度

输入: 偏心率 e , 轨道角动量 h , 轨道倾角 i , 真近点角 θ , 升交点赤经 Ω , 近地点幅角 ω

输出: 三维位置向量 (r_x, r_y, r_z) , 三维速度向量 (v_x, v_y, v_z)

- 1: 计算轨道长半轴 $a = \frac{h^2}{GM(1-e^2)}$
 - 2: 计算卫星至地心的距离 $r = \frac{a(1-e^2)}{1+e \cos(\theta)}$
 - 3: 计算半通径 $p = a(1 - e^2)$
 - 4: 计算方向向量 $\mathbf{P}$ 和 $\mathbf{Q}$
 - 5: 计算三维位置向量 $\mathbf{r}_{\text{vector}} = r \cos(\theta)\mathbf{P} + r \sin(\theta)\mathbf{Q}$
 - 6: 计算三维速度向量 $\mathbf{v}_{\text{vector}} = -\frac{h}{p} \sin(\theta)\mathbf{P} + \frac{h}{p}(e + \cos(\theta))\mathbf{Q}$
 - 7: 输出 $\mathbf{r}_{\text{vector}} = (r_x, r_y, r_z)$ 和 $\mathbf{v}_{\text{vector}} = (v_x, v_y, v_z)$
-

算法 2 从位置和速度求轨道根数

输入: 三维位置向量 (r_x, r_y, r_z) , 三维速度向量 (v_x, v_y, v_z)

输出: 偏心率 e , 轨道角动量 h , 轨道倾角 i , 升交点赤经 Ω , 近地点幅角 ω , 真近点角 θ

- 1: 计算位置模长 $r = \|\mathbf{r}_{\text{vector}}\|$ 和速度模长 $v = \|\mathbf{v}_{\text{vector}}\|$
 - 2: 计算角动量向量 $\mathbf{h}_{\text{vector}} = \mathbf{r}_{\text{vector}} \times \mathbf{v}_{\text{vector}}$
 - 3: 计算角动量 $h = \|\mathbf{h}_{\text{vector}}\|$
 - 4: 计算轨道倾角 $i = \arctan\left(\frac{\sqrt{h_x^2 + h_y^2}}{h_z}\right)$
 - 5: 计算升交点赤经 $\Omega = \arctan\left(\frac{h_x}{-h_y}\right)$
 - 6: 计算半通径 $p = \frac{h^2}{GM}$
 - 7: 计算轨道长半轴 $a = \left(\frac{2}{r} - \frac{v^2}{GM}\right)^{-1}$
 - 8: 计算偏心率 $e = \sqrt{1 - \frac{p}{a}}$
 - 9: 计算偏近点角 $E = \arctan\left(\frac{(\mathbf{r}_{\text{vector}} \cdot \mathbf{v}_{\text{vector}})}{a^2 n}, 1 - \frac{r}{a}\right)$
 - 10: 计算平近点角 $M = E - e \sin(E)$
 - 11: 计算纬度幅角 $u = \arctan\left(\frac{r_z}{-r_x h_y + r_y h_x}\right)$
 - 12: 计算真近点角 $\theta = \arctan\left(\frac{\sqrt{1-e^2} \sin(E)}{\cos(E) - e}\right)$
 - 13: 计算近地点幅角 $\omega = u - \theta$
 - 14: 输出 $e, h, \Omega, i, \omega, \theta$
-

在本文中, 通过反推轨道根数这一过程, 可检验初始轨道根数解算出的位置和速度的准确性, 从而评估整个模型的可靠性, 算法实现流程如上表所示。

首先, 通过位置和速度向量, 可以直接计算出卫星的角动量向量。计算出角动量向量后, 借助于向量的几何关系, 可进一步求解出轨道的倾角和升交点赤经。这两个参数会直接影响到轨道平面的确定。

其次，使用能量守恒和开普勒定律计算出的长半轴和偏心率长半轴与卫星轨道的大小相关，而偏心率则描述轨道形状的偏离圆形的程度。

然后，通过位置向量和速度向量的几何关系计算偏近点角 E ，利用开普勒方程平近点角 M ，进而计算真近点角，并基于轨道的几何关系计算纬度幅角，最后求解近地点幅角。

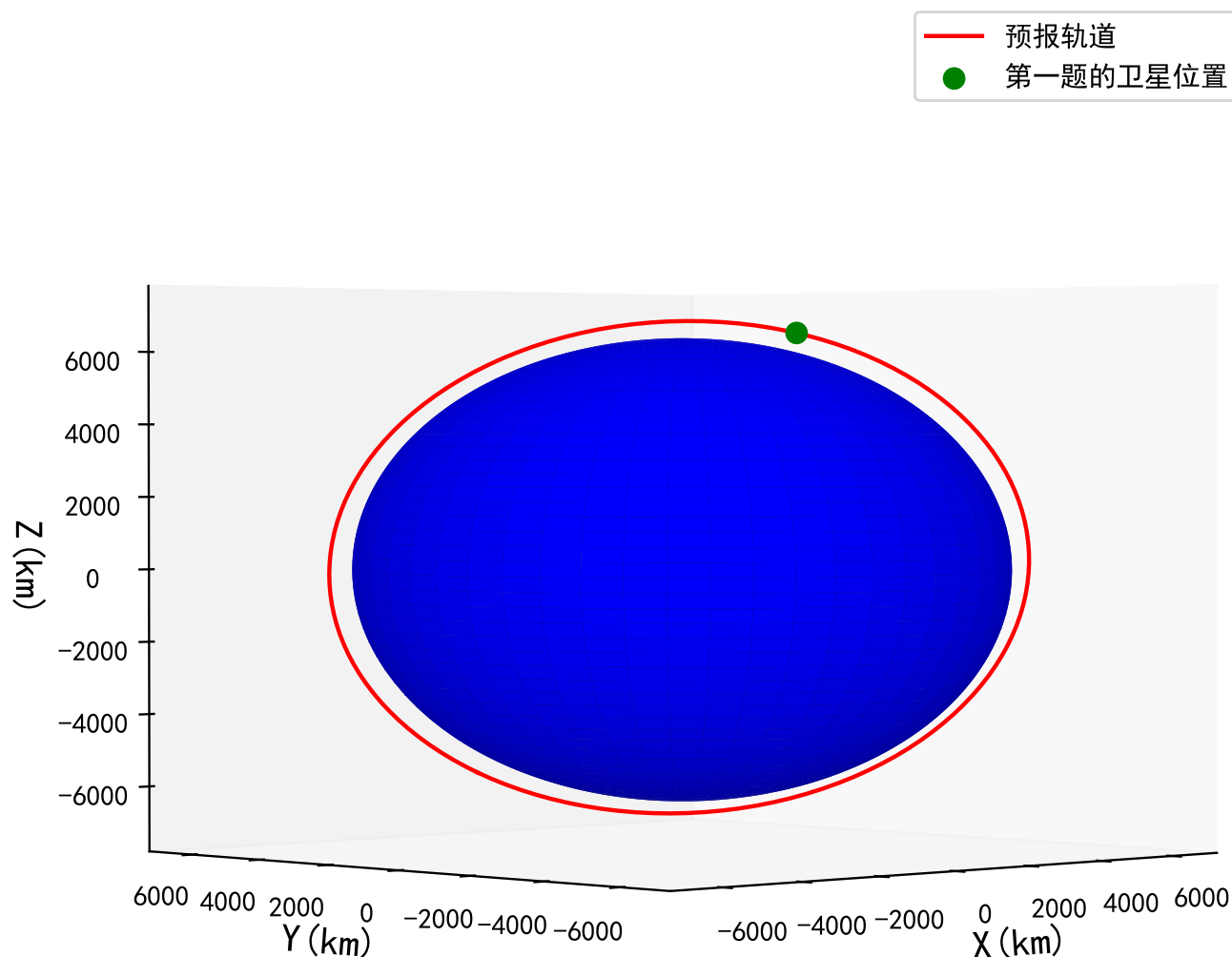


图 3.5 求解得到的卫星轨道示意图

如图3.5所示是根据题目所给的六个开普勒轨道根数求解得到的三维卫星轨道，图中的中心蓝色圆球为地球，绿色点为所解算得到的卫星位置，红线为根据真近地点角 θ 从 $0-2\pi$ 变化所预报得到的一个完整周期的卫星轨道。由于在六个开普勒轨道根数中，除了 θ 是随着卫星不断运动而发生变化外，其余五个轨道根数均与时间无关，因此可以通过改变 θ 的数值来模拟对卫星轨道进行预报的过程。从图3.5中的结果可以看出，当 θ 改变时，由于不考虑摄动力以及第三体引力等因素的影响，卫星的轨道变化得十分平稳和光滑，表明了本文所建立了的二体运动卫星轨道求解模型的轨道参数具有高度一致性。同时本文通过比较了当设置 θ 为 0 和 2π 时，所分别解算出半长轴之和刚好等于整个轨道的的长半轴，有效证明了本文所采用的模型的准确性。

表 3.1 由轨道根数求解得到的卫星的位置和速度

参数	X 方向	Y 方向	Z 方向
位置坐标 (km)	1274.9134151	-1848.8519650	6507.2626553
速度 (km/s)	-6.2107951	3.7461273	2.2763309

表 3.2 由位置和速度计算的轨道根数

轨道根数	数值
偏心率 e	$2.06136076 \times 10^{-3}$
角动量 h	$5.23308462 \times 10^4 \text{ km}^2/\text{s}$
轨道倾角 i	1.69931232 rad
真近点角 θ	3.43807372 rad
升交点赤经 Ω	5.69987423 rad
近地点幅角 ω	4.10858621 rad

表3.1为根据六个轨道根数求解得到的卫星位置和速度结果，为了验证计算结果的准确性和可靠性，本文通过将解算得到的位置和速度反算开普勒轨道根数，结果如表3.2所示。反算得到的轨道根数数值与题目所提供的轨道根数数值完全相等，进一步有力地验证了本文所建模型的轨道参数的一致性、位置、速度结果的准确性。

4 问题二的建模与求解

针对问题二的要求，本文制定了模型的建立与求解框架如图4.1所示，在问题二中，由于考虑的是理想真空环境，且 Crab 脉冲星距离太阳系相当远（约 6500 光年），因此可以假设 X 射线光子信号为平行光且忽略太阳系天体的自转和扁率。由于卫星采用的是地球质心为原点的地球天球参考系，而脉冲星的星历参数均是定义在以太阳系质心为原点的天球坐标系，因此在求解的时候需要考虑到坐标系的转换。此外由于地球时与太阳系质心力学时存在差异，为了精确计算时延同样需要考虑时间系统的转换。

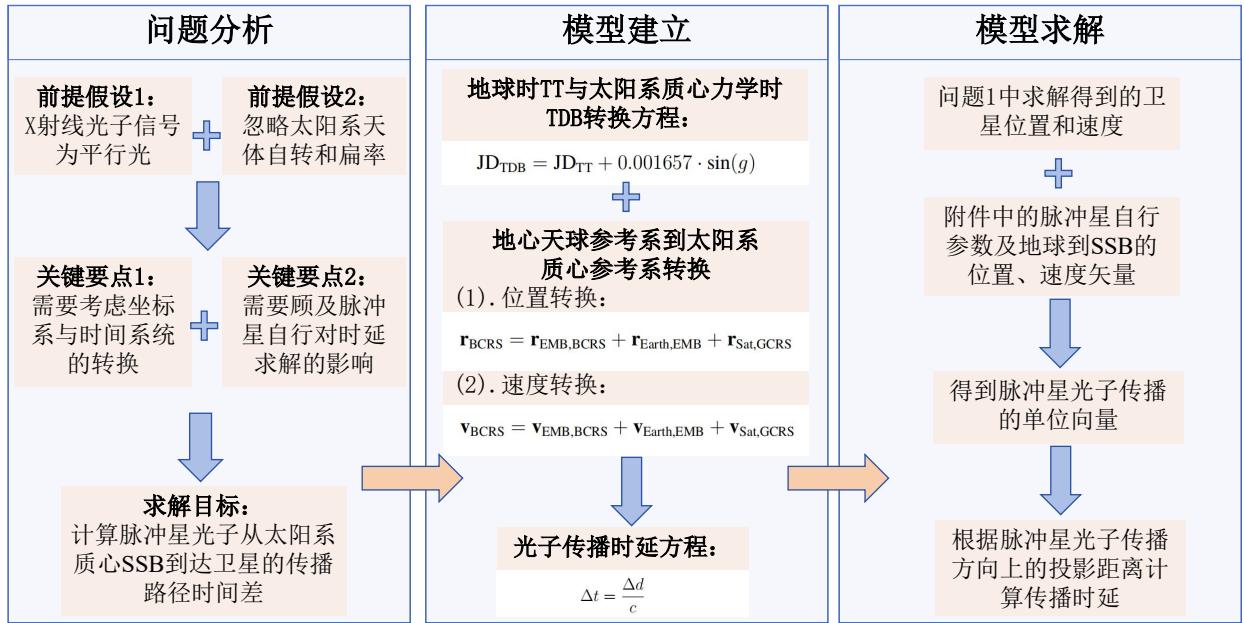


图 4.1 问题二流程图

基于上述对问题二的分析，本文通过构建时间系统转换方程、坐标系统转换方程，并结合光子传播路径差的性质建立脉冲光子传播的时延模型。首先利用问题一的计算结果（卫星位置和速度）对模型进行求解，然后由自行参数计算脉冲星光子传播的单位向量，进而得到其方向上的投影距离及传播时延。

4.1 问题分析

根据本题目描述，假设脉冲星发出的 X 射线光子信号为平行光，忽略太阳系天体的自转和扁率。题目要求计算脉冲星光子从太阳系质心 SSB 到达卫星的传播路径时间差，该时间差是由卫星和太阳系几何路径差引起的。图4.2为脉冲星相对于 SSB 和航天器的位置关系， $\vec{r}$ 为从太阳系质心 SSB 到航天器的矢量， $\vec{r}_E$ 为从 SSB 到地球质心的矢量， $\vec{r}_E^S$ 为从地球质心到航天器的矢量， d 为脉冲星到太阳系质心 SSB 的距离。

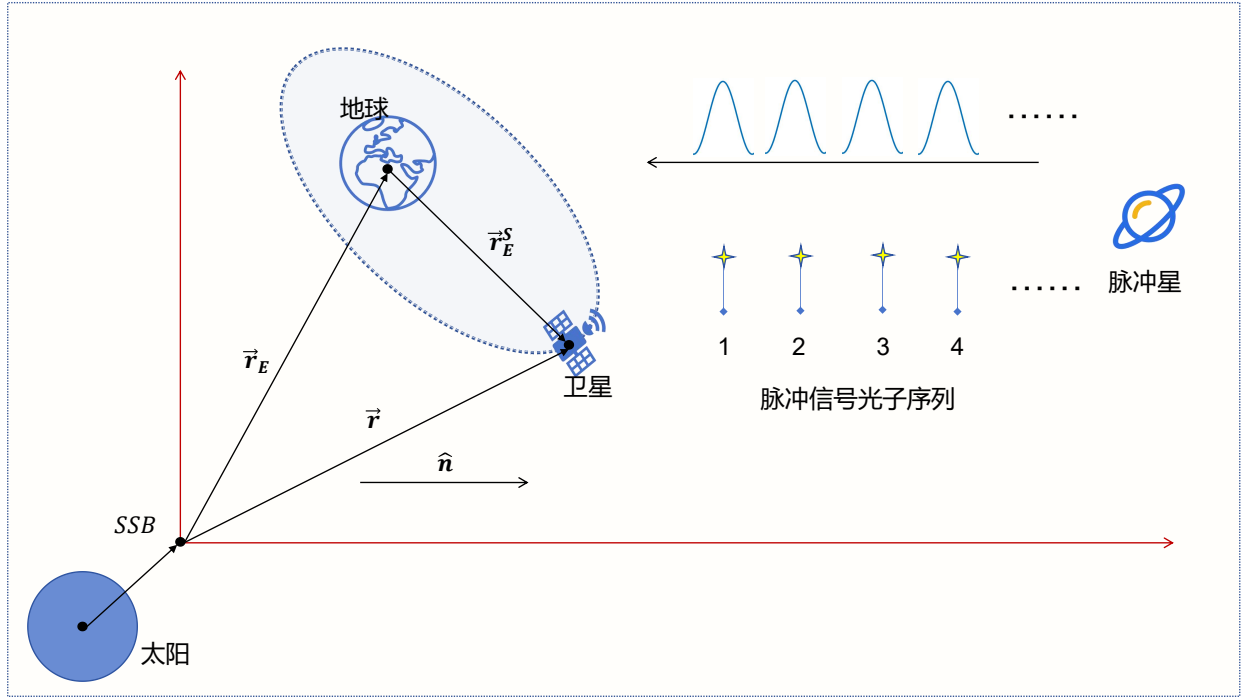


图 4.2 脉冲星和其他天体位置关系

4.2 模型建立

在本问题中，需建立脉冲星光子到达太阳系质心 SSB 与到达卫星的传播时延差模型。以下为模型的详细建立过程：

(1) 时间的表示与转换

在该光子到达时间模型中，时间采用儒略日 (Julian Date, JD) 进行表示。而卫星的观测时间以约化儒略日 (Modified Julian Date, MJD) 给出。MJD 与 JD 之间的关系为：

$$JD = MJD + 2400000.5 \quad (4.1)$$

然而，为满足脉冲星导航的高精度需求，还需要将地球时 TT 时间系统转换为太阳系质心力学时 TDB 时间系统。不同时间系统的转换关系如图4.3所示。

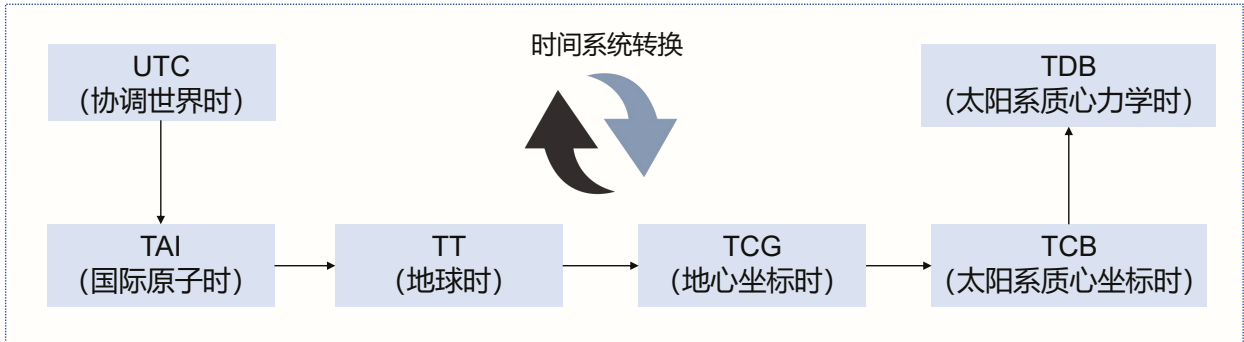


图 4.3 时间系统转换

图4.3中的时间转换可以从日常使用最频繁的协调世界时 (UTC) 开始, 经过国际原子时 (TAI)、地球时 (TT)、地心坐标时 (TCG) 以及太阳系质心坐标时 (TCB), 最后得到太阳系质心力学时 (TDB)。整个时间转换过程可逆, 也可以直接从中间某一时间系统出发转换到其他时间系统。不同时间尺度之间的转换公式如下:

$$\text{TAI} = \text{UTC} + \Delta t \quad (4.2)$$

$$\text{TT} = \text{TAI} + 32.184 \text{ s} + \delta \quad (4.3)$$

$$\text{TCG} - \text{TT} = \frac{L_G}{1 - L_G} (J_{\text{TT}} - T_0) \times 86400 \text{ s} \quad (4.4)$$

$$\text{TCB} - \text{TCG} = \frac{1}{c^2} \int \left[U_B + \frac{v_B^2}{2} + \Delta L_C^{(PN)} + \Delta L_C^{(A)} \right] dt \quad (4.5)$$

公式中的常数为: $L_G \equiv 6.969290134 \times 10^{-10}$, $L_B \equiv 1.550519768 \times 10^{-8}$, $T_0 = 244\,314\,4.500\,372\,5$, $B_0 \equiv -6.55 \times 10^{-5} \text{ s}$, $\Delta L_C^{(PN)} = 1.097 \times 10^{-16}$, $\Delta L_C^{(A)} = 5 \times 10^{-18}$ 。

结合上文所述, 本题目只需要从 TT 转换到 TDB 需实现的时间转换公式如下:

$$\text{JD}_{\text{TDB}} = \text{JD}_{\text{TT}} + 0.001657 \cdot \sin(g) \quad (4.6)$$

其中, $g = 6.24 + 0.017202 \cdot (\text{JD}_{\text{TT}} - 2451545)$ 。

(2) 坐标转换: 从地心天球参考系到太阳系质心参考系

问题一中求解得到的卫星位置是在地心天球参考系 GCRS 下的坐标, 然而为了进行准确的时延建模, 需要将卫星的 GCRS 坐标转换为太阳系质心参考系 BCRS 坐标。

通过从 JPL 行星历表中获取地月质心相对于太阳系质心的位置向量和地球质心相对于地月质心的位置向量, 然后将其与卫星在 GCRS 中的位置向量相加, 得到卫星在 BCRS 中的位置向量即实现了卫星的 GCRS 坐标到太阳系质心参考系 BCRS 坐标的转换。具体的坐标转换公式如下:

$$\mathbf{r}_{\text{BCRS}} = \mathbf{r}_{\text{EMB,BCRS}} + \mathbf{r}_{\text{Earth,EMB}} + \mathbf{r}_{\text{Sat,GCRS}} \quad (4.7)$$

其中, $\mathbf{r}_{\text{EMB,BCRS}}$ 是地月质心在太阳系质心 BCRS 中的位置, $\mathbf{r}_{\text{Earth,EMB}}$ 是地球质心相对于地月质心的位置, $\mathbf{r}_{\text{Sat,GCRS}}$ 是卫星在 GCRS 中的位置, $\mathbf{r}_{\text{BCRS}}$ 是卫星在 BCRS 中的位置。

同理, 速度转换公式为:

$$\mathbf{v}_{\text{BCRS}} = \mathbf{v}_{\text{EMB,BCRS}} + \mathbf{v}_{\text{Earth,EMB}} + \mathbf{v}_{\text{Sat,GCRS}} \quad (4.8)$$

(3) 脉冲星光子传播方向的确定

脉冲星的位置通过赤经 (α) 和赤纬 (δ) 来描述。由于脉冲星的自行, 赤经和赤纬会随时间发生微小变化。图4.4为脉冲星自行影响示意图:

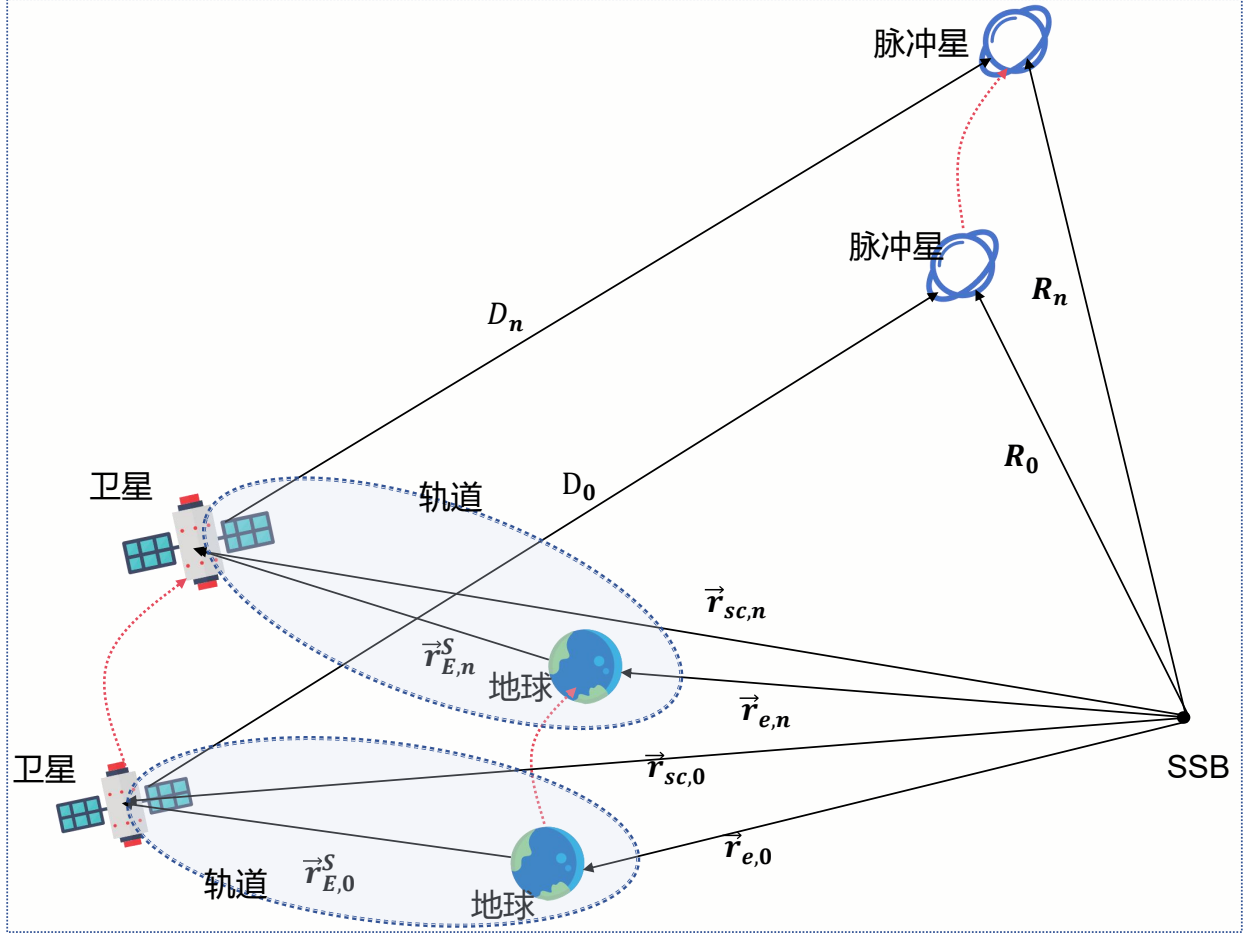


图 4.4 脉冲星自行影响

其中 D_0 为参考时刻航天器至卫星的矢量, D_n 为解求时刻航天器至卫星的矢量, R_0 为参考时刻 SSB 至脉冲星的矢量, R_n 为解求时刻 SSB 至脉冲星的矢量, $\vec{r}_{sc}$ 为 SSB 至卫星的矢量, $\vec{r}_E$ 为地球至卫星的矢量, $\vec{r}_e$ 为 SSB 至地球的矢量, 在实际结算时需使用自行参数对其进行修正以获取该时刻的赤经和赤纬。脉冲星的赤经和赤纬修正公式表示如下:

$$\alpha = \alpha_{\text{ref}} + \dot{\alpha} \cdot (t - t_{\text{ref}}) \quad (4.9)$$

$$\delta = \delta_{\text{ref}} + \dot{\delta} \cdot (t - t_{\text{ref}}) \quad (4.10)$$

确定赤经和赤纬后, 可得到脉冲星光子在 SSB 中的传播方向单位向量为:

$$u_{\text{pulsar}} = \begin{pmatrix} \cos \delta \cdot \cos \alpha \\ \cos \delta \cdot \sin \alpha \\ \sin \delta \end{pmatrix} \quad (4.11)$$

(4) 几何传播时延的计算

几何传播时延是通过卫星位置在脉冲星光子传播方向上的投影计算的。公式如下：

$$\Delta d = r_{\text{SSB}} \cdot u_{\text{pulsar}} \quad (4.12)$$

(5) 时延的计算

传播路径差 Δd 转化为传播时延 Δt 的公式如下：

$$\Delta t = \frac{\Delta d}{c} \quad (4.13)$$

通过这些步骤，可计算脉冲星光子到达卫星与太阳系质心的传播路径时间差。

4.3 模型求解与结果分析

在模型建立的基础上，进行脉冲星光子几何传播时延的求解过程。以下是求解的主要步骤：

(1) 时间表示与时间系统转换

卫星的观测时间通常以儒略日 (Julian Date, JD) 表示，而在计算过程中，涉及到的时间系统转换需要从地球时间系统 TT (Terrestrial Time) 转换为太阳系质心参考系下的时间 TDB (Barycentric Dynamical Time)。根据式 (4.6) 计算得到 TDB 时间：

$$\text{JD}_{\text{TDB}} = 2457062.5 \quad (4.14)$$

该 TDB 时间是卫星在太阳系质心下的观测时间。

(2) 转换卫星在 GCRS 系中的位置至 BCRS 系

根据问题一所解算的结果，卫星在 GCRS 系中的三维坐标为：

$$r_{\text{Sat,GCRS}} = (1274.9134151 \text{ km}, -1848.8519650 \text{ km}, 6507.2626553 \text{ km}) \quad (4.15)$$

通过 JPL 行星历表 (DE 系列) 查询，地月质心相对于太阳系质心 SSB 的位置和地球质心相对于地月质心的位置分别为：

$$r_{\text{EMB,BCRS}} = (-112095835.3780862 \text{ km}, 87510403.8439392 \text{ km}, 37914349.6454615 \text{ km}) \quad (4.16)$$

$$r_{\text{Earth,EMB}} = (4745.4780111 \text{ km}, 1115.2437756 \text{ km}, 455.0556418 \text{ km}) \quad (4.17)$$

其中， $r_{\text{Earth,EMB}}$ 是地球质心相对于地月质心的位置向量。

因此，地球质心在 BCRS 系中的位置可以表示为：

$$r_{\text{Earth,BCRS}} = r_{\text{EMB,BCRS}} + r_{\text{Earth,EMB}} \quad (4.18)$$

通过向量相加，得到地球质心的精确位置：

$$r_{\text{Earth,BCRS}} = (-112091089.9000751 \text{ km}, 87511519.0877148 \text{ km}, 37914804.7011034 \text{ km}) \quad (4.19)$$

接下来，将卫星的位置从 GCRS 系转换到 BCRS 系，考虑地月质心影响的坐标转换公式如下：

$$r_{\text{BCRS}} = r_{\text{Earth,BCRS}} + r_{\text{Sat,GCRS}} \quad (4.20)$$

通过位置坐标相加，得到卫星在 BCRS 系中的位置向量为：

$$r_{\text{BCRS}} = (-112089814.9866600 \text{ km}, 87509670.2357498 \text{ km}, 37921311.9637586 \text{ km}) \quad (4.21)$$

(3) 计算脉冲星光子传播的单位向量

根据式 (4.10)，对脉冲星的赤经 (α) 和赤纬 (δ) 进行修正，得到观测时刻值为：

$$\alpha = 83.63308361364257^\circ, \quad \delta = 22.014492275921643^\circ \quad (4.22)$$

计算脉冲星光子在太阳系质心参考系 (BCRS) 中传播方向的单位向量：

$$u_{\text{pulsar}} = \begin{pmatrix} \cos \delta \cdot \cos \alpha \\ \cos \delta \cdot \sin \alpha \\ \sin \delta \end{pmatrix} = \begin{pmatrix} 0.10280982 \\ 0.92137086 \\ 0.37484113 \end{pmatrix} \quad (4.23)$$

(4) 计算几何传播时延

几何传播时延的计算是基于卫星位置在脉冲星光子传播方向上的投影距离，根据式 (4.12)，计算得到投影距离：

$$\Delta d_{\text{geo}} = -83319417239.6452 \text{ km} \quad (4.24)$$

将传播路径差 Δd_{geo} 转化为几何传播时延：

$$\Delta t_{\text{geo}} = \frac{\Delta d_{\text{geo}} \times 10^3}{c} = \frac{-83319417239.6452026 \times 10^3}{299792458} \text{ s} \quad (4.25)$$

通过计算得到几何传播时延为：

$$\Delta t_{\text{geo}} = -277.92366023979565 \text{ s} \quad (4.26)$$

综上所述，问题二考虑了坐标系和时间系统的转换，并且顾及脉冲星自行的影响，最终求解得到脉冲星光子到达卫星与太阳系质心的传播路径时间差为

$$\Delta t_{\text{geo}} = -277.92366023979565 \text{ s}$$

5 问题三的建模与求解

针对问题三的要求，本文制定了整体的模型建立与求解框架如图5.1所示。与问题二类似，在问题三中也需假设 X 射线为平行光以及忽略太阳系天体自转和扁率的影响，但在问题二的基础上需要考虑对 Shapiro 时延、引力红移时延和狭义相对论的动钟变慢效应对脉冲光子传播时延的影响，并精确对其进行建模与求解。

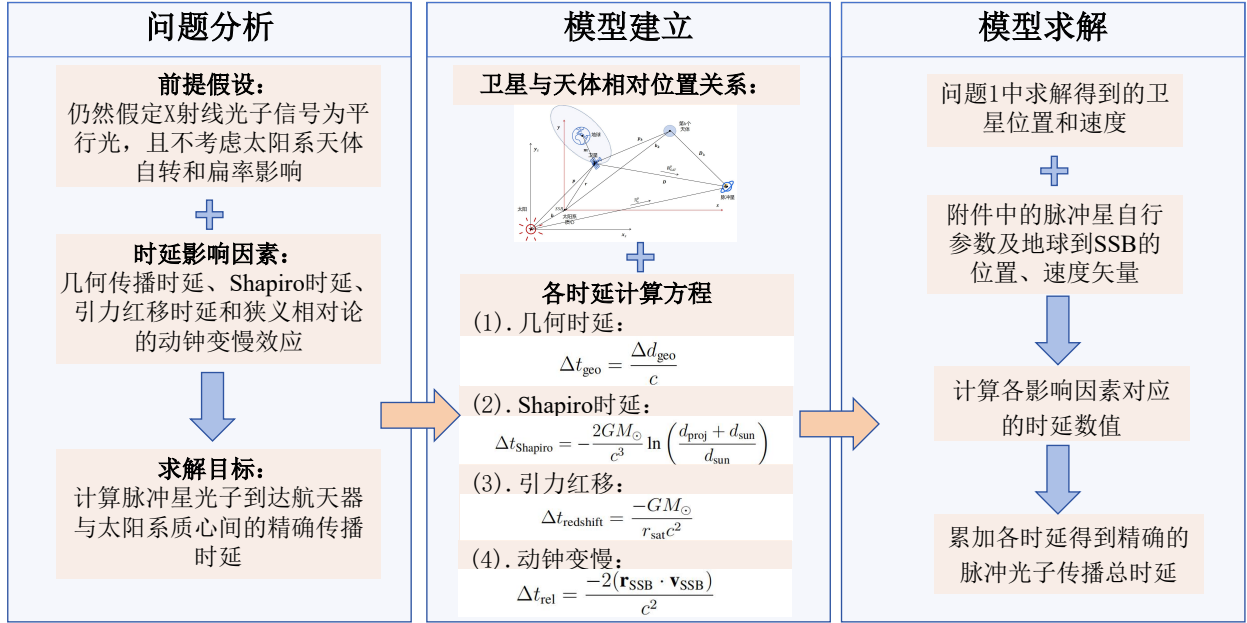


图 5.1 问题三流程图

为了计算脉冲光子到达航天器与太阳系质心间的精确传播时延，本文考虑卫星与其他天体的相对位置关系，并对 Shapiro 时延、引力红移时延和狭义相对论的动钟变慢效应的产生原因建立精确的时延转换模型并求解，最终获得精确的脉冲光子传播时延差。

5.1 问题分析

根据本题目描述，脉冲星光子从脉冲星发出后，经过太阳系内的引力场和其他时空效应后到达卫星，需要考虑多个时延因素。首先，脉冲星发出的 X 射线光子信号假设为平行光，忽略太阳系天体的自转和扁率对光子传播路径的影响。题目要求建立脉冲星光子到达卫星与太阳系质心 SSB（Solar System Barycenter）之间的精确转换时延模型，主要考虑以下几个关键的时延因素：

1. **几何传播时延：**脉冲星光子从脉冲星发出后，经过有限的传播时间才到达卫星。几何传播时延是由于光子传播过程中，卫星的几何位置与太阳系质心不同，存在传播路径差异。这一时延根据光子传播路径上的几何投影距离来计算。

2. **Shapiro 时延：**Shapiro 时延是由太阳引力场引起的光子路径弯曲效应。由于太阳是主要的引力源，光子经过太阳附近时，其传播路径会发生弯曲，导致光子到达卫星的时间

延迟。Shapiro 时延与太阳相对卫星的距离以及光子传播方向相关。

3. **引力红移时延**：引力红移是由于光子在强引力场中传播时，频率发生变化而产生的时间效应。在太阳系范围内，主要是太阳的引力造成了光子的频率偏移，这对光子到达卫星的时间产生了一定的延迟。

4. **狭义相对论的动钟效应**：根据狭义相对论，运动物体的时间会变慢，尤其是当物体的速度接近光速时。这一效应会影响到卫星的时间上，导致光子到达时间的偏差。需考虑卫星在太阳系质心参考系中的速度对时延的影响。

5. **脉冲星自行影响**：脉冲星的赤经和赤纬随着时间发生变化，称为脉冲星的自行效应。在时延模型中，必须修正脉冲星的自行，确保在给定观测时刻计算出精确的脉冲星位置，从而准确计算光子传播方向。

通过考虑以上几个时延因素，模型的目标是精确计算光子从脉冲星发出后，到达卫星与太阳系质心之间的时间延迟。根据给定的光子到达探测器的时刻，并结合卫星的位置、速度和脉冲星的自行参数信息，建立综合的时延模型。

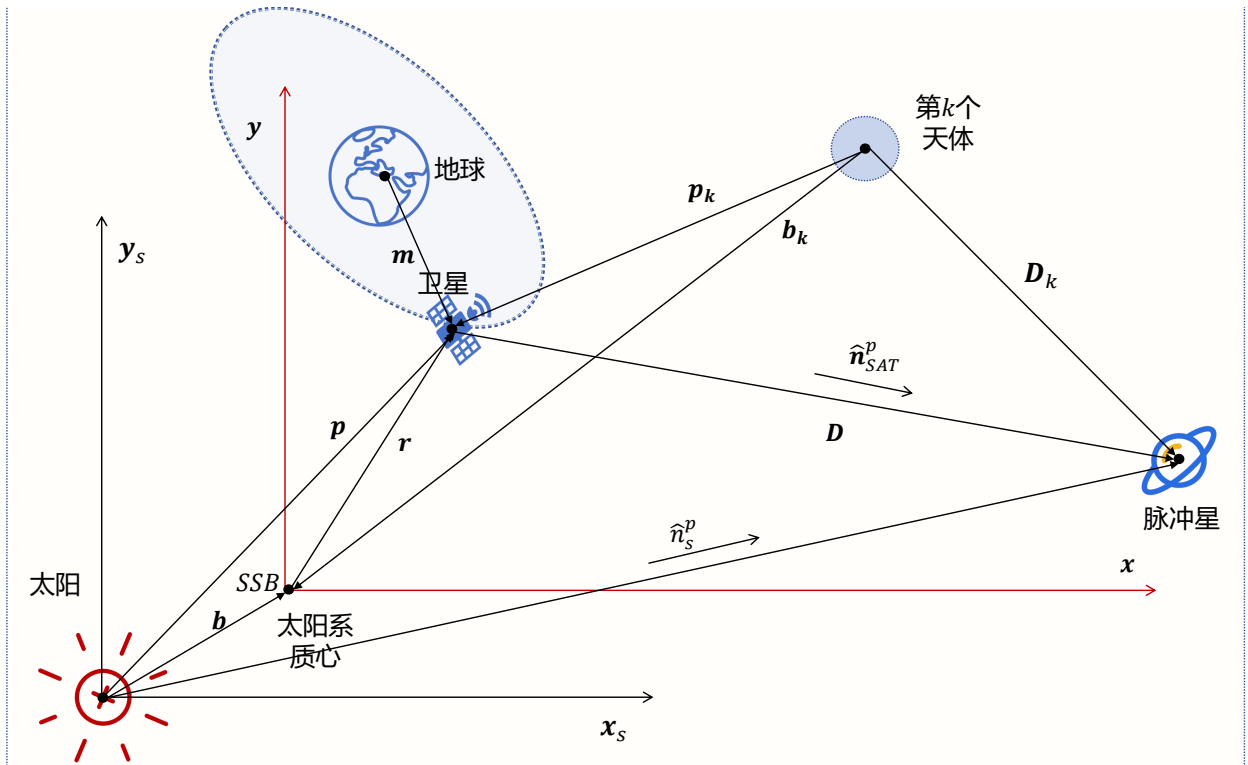


图 5.2 脉冲星与各天体相对位置

5.2 模型建立

在本问题中，需建立脉冲星光子从太阳系质心 SSB 到达卫星的精确时延转换模型，除了几何传播时延外，还需要考虑 Shapiro 时延、引力红移时延和狭义相对论动钟效应等关键因素。为了便于讨论到达时间的延迟关系，用某个光子代替整个脉冲的传播过程，以脉

冲星到探测器方向作为坐标系的 X 轴重新描述光子的传播如图5.2。其中 $\mathbf{D}$ 和 $\mathbf{p}$ 分别为从太阳质心到脉冲星和航天器的矢量， $\mathbf{D}_k$ 和 $\mathbf{p}_k$ 分别为从太阳系内第 k 个天体到脉冲星和航天器的矢量， $\hat{\mathbf{n}}_p$ 为从航天器到脉冲星的单位矢量， $\hat{\mathbf{n}}_s$ 为从太阳中心到脉冲星的单位矢量。

下面是脉冲星光子到达航天器与太阳系质心的精确转换模型的详细建立过程：

(1) 时间系统转换

在本问题考虑时延的精确建模中，同样需将 TT 时间转换为 TDB 时间。TT 时间和 TDB 时间之间的转换公式同式 (4.6)。

(2) 坐标转换：从地心天球参考系到太阳系质心参考系

问题一中的卫星位置和速度是在地心天球参考系 (GCRS) 下给出的，因此，类似于问题二，本题目同样需将卫星的 GCRS 坐标转换为太阳系质心参考系 BCRS。从 JPL 行星历表中获取地月质心相对于太阳系质心的位置向量，以及地球质心相对于地月质心的位置向量，然后将其与卫星在 GCRS 中的位置和速度相加，得到卫星在 BCRS 中的坐标，转换公式同式 (4.7)。同理，速度的转换公式为：

$$\mathbf{V}_{\text{BCRS}} = \mathbf{V}_{\text{EMB,BCRS}} + \mathbf{V}_{\text{Earth,EMB}} + \mathbf{V}_{\text{Sat,GCRS}} \quad (5.1)$$

(3) 脉冲星光子传播方向的确定

脉冲星的赤经 (α) 和赤纬 (δ) 随时间会发生变化，因此使用脉冲星自行参数对其进行修正。脉冲星传播方向的单位向量表示同式 (4.11)：

(4) 几何传播时延的计算

几何传播时延由卫星位置在脉冲星光子传播方向上的投影计算得出 Δt_{geo} ：

$$\Delta t_{\text{geo}} = \frac{\Delta d_{\text{geo}}}{c} \quad (5.2)$$

(5) Shapiro 时延的计算

Shapiro 时延考虑了光子经过大质量天体时由于引力场造成的时延，由于太阳是太阳系内主要的引力源，因此本研究只考虑太阳引力造成的延迟。那么 Shapiro 时延计算公式为：

$$\Delta t_{\text{Shapiro}} = -\frac{2GM_{\odot}}{c^3} \ln \left(\frac{d_{\text{proj}} + d_{\text{sun}}}{d_{\text{sun}}} \right) \quad (5.3)$$

其中， d_{proj} 为卫星相对于太阳在脉冲星光子方向上的投影距离， d_{sun} 为卫星与太阳的距离。

(6) 引力红移时延的计算

引力红移时延由卫星处于太阳引力势场中引起，公式如下：

$$\Delta t_{\text{redshift}} = \frac{-GM_{\odot}}{r_{\text{sat}}c^2} \quad (5.4)$$

其中， r_{sat} 是卫星与太阳的距离。

(7) 狭义相对论时延的计算

狭义相对论动钟效应考虑了卫星的运动速度对时间流逝的影响，计算公式为：

$$\Delta t_{\text{rel}} = \frac{-2(\mathbf{r}_{\text{SSB}} \cdot \mathbf{v}_{\text{SSB}})}{c^2} \quad (5.5)$$

(8) 总时延的计算

最终的时延由几何传播时延、Shapiro 时延、引力红移时延和狭义相对论时延共同组成：

$$\Delta t_{\text{total}} = \Delta t_{\text{geo}} + \Delta t_{\text{Shapiro}} + \Delta t_{\text{redshift}} + \Delta t_{\text{rel}} \quad (5.6)$$

通过这些步骤，能够精确计算脉冲星光子到达卫星的总时延。

5.3 模型求解与结果分析

本节在所建立精确转换时延模型基础上，进行脉冲星光子传播时延的求解过程，几何时延求解过程与问题二一致，本节不再过多赘述，本问题求解步骤如下：

(1) 时间系统转换

同问题二，本题目给出的时间是 TT（平太阳时）系统的时间，因此需将其转换为 TDB（太阳系质心力学时）时间系统，以确保时间与 BCRS 系一致。根据式 (4.6)，完成时间系统的转换。

(2) 转换卫星在 GCRS 系中的位置至 BCRS 系

根据问题一所解算的结果，卫星在 GCRS 系中的三维坐标同式 (4.15)，通过 JPL 行星历表（DE 系列）查询，地月质心相对于太阳系质心 BCRS 的位置和地球质心相对于地月质心的位置分别为：

$$\mathbf{r}_{\text{EMB,BCRS}} = (-26368299.6290212 \text{ km}, 133585300.4683054 \text{ km}, 57890210.3794789 \text{ km}) \quad (5.7)$$

$$\mathbf{r}_{\text{Earth,EMB}} = (-222.0379084 \text{ km}, -4083.4326476 \text{ km}, -1451.1329569 \text{ km}) \quad (5.8)$$

通过向量相加，得到地球质心的位置：

$$\mathbf{r}_{\text{Earth,BCRS}} = (-26368521.6669296 \text{ km}, 133581217.0356578 \text{ km}, 57895266.5091774 \text{ km}) \quad (5.9)$$

接下来，将卫星的位置从 GCRS 系转换到 BCRS 系通过位置坐标相加，得到卫星在 BCRS 系中的位置向量为：

$$\mathbf{r}_{\text{BCRS}} = (-26367246.7535145 \text{ km}, 133579368.1836928 \text{ km}, 57895266.5091774 \text{ km}) \quad (5.10)$$

同理，卫星在 BCRS 系中的速度向量为：

$$\mathbf{v}_{\text{BCRS}} = (-35.9883897 \text{ km/s}, -1.2969601 \text{ km/s}, 0.0896335 \text{ km/s}) \quad (5.11)$$

(3) 计算脉冲星光子传播的单位向量

根据式 (4.10)，对脉冲星的赤经 (α) 和赤纬 (δ) 进行修正，得到观测时刻的赤经和赤纬为：

$$\alpha = 83.6330717947614^\circ, \quad \delta = 22.014493883932683^\circ \quad (5.12)$$

计算脉冲星光子在太阳系质心参考系 (BCRS) 中传播方向的单位向量：

$$\mathbf{u}_{\text{pulsar}} = \begin{pmatrix} \cos \delta \cdot \cos \alpha \\ \cos \delta \cdot \sin \alpha \\ \sin \delta \end{pmatrix} = \begin{pmatrix} 0.10280982 \\ 0.92137086 \\ 0.37484113 \end{pmatrix} \quad (5.13)$$

(4) 计算几何传播时延

几何传播时延的计算是基于卫星位置在脉冲星光子传播方向上的投影距离，根据式 (4.12)，计算得到投影距离：

$$\Delta d_{\text{geo}} = -142066853076.3861084 \text{ km} \quad (5.14)$$

将传播路径差 Δd_{geo} 转化为几何传播时延：

$$\Delta t_{\text{geo}} = \frac{\Delta d_{\text{geo}} \times 10^3}{c} = \frac{-142066853076.3861084 \times 10^3}{299792458} \text{ s} \quad (5.15)$$

通过计算，得到几何传播时延为：

$$\Delta t_{\text{geo}} = -473.88401304073534 \text{ s} \quad (5.16)$$

(4) Shapiro 时延的计算

Shapiro 时延由光子经过太阳时的引力场产生，计算公式同式 (5.3)，其中， $GM_\odot = 1.327 \times 10^{20} \text{ m}^3/\text{s}^2$ 。代入公式后得到：

$$\Delta t_{\text{Shapiro}} = -6.6254740275614095 \times 10^{-6} \text{ s} \quad (5.17)$$

(5) 引力红移时延的计算

由于卫星处于太阳的引力势场之中，因此引力红移时延由式 (5.4) 可以计算：代入 $r_{\text{sat}} = 147954524060.2123718 \text{ km}$ ，计算得，

$$\Delta t_{\text{redshift}} = -9.979327322105222 \times 10^{-9} \text{ s} \quad (5.18)$$

(6) 狭义相对论动钟效应的计算

狭义相对论时延考虑了卫星的运动速度对时间流逝的影响，计算公式为式 (5.5)，经过计算，得到：

$$\Delta t_{\text{rel}} = -7.027050984279227 \times 10^{-10} \text{ s} \quad (5.19)$$

(7) 总时延的计算

综合几何传播时延、Shapiro 时延、引力红移时延和狭义相对论动钟效应的计算结果，总时延为：

$$\begin{aligned}\Delta t_{\text{total}} = & -473.88401304073534 \text{ s} - 6.6254740275614095 \times 10^{-6} \text{ s} \\ & - 9.979327322105222 \times 10^{-9} \text{ s} - 7.027050984279227 \times 10^{-10} \text{ s}\end{aligned}\quad (5.20)$$

各时延的具体数值如表5.1所示，最终计算的总时延结果为：

$$\Delta t_{\text{total}} = -473.8840196768914 \text{ s} \quad (5.21)$$

表 5.1 不同因素引起的时间延迟

时延因素	延迟 (s)
几何延迟	-473.88401304073534
Shapiro 效应延迟	$-6.6254740275614095 \times 10^{-6}$
红移效应延迟	$-9.979327322105222 \times 10^{-9}$
狭义相对论延迟	$-7.027050984279227 \times 10^{-10}$
总延迟	-473.8840196768914

6 问题四的建模与求解

针对问题四的要求，本文采用的模型建立与解算框架如图6.1所示。由于已知条件归一化后的标准脉冲轮廓曲线数据和 Crab 脉冲星的自转参数及流量密度数据，因此本题的关键要点是仿真出的光子脉冲序列需要满足非齐次泊松分布，且折叠后的轮廓需要和标准轮廓具有良好相关关系。

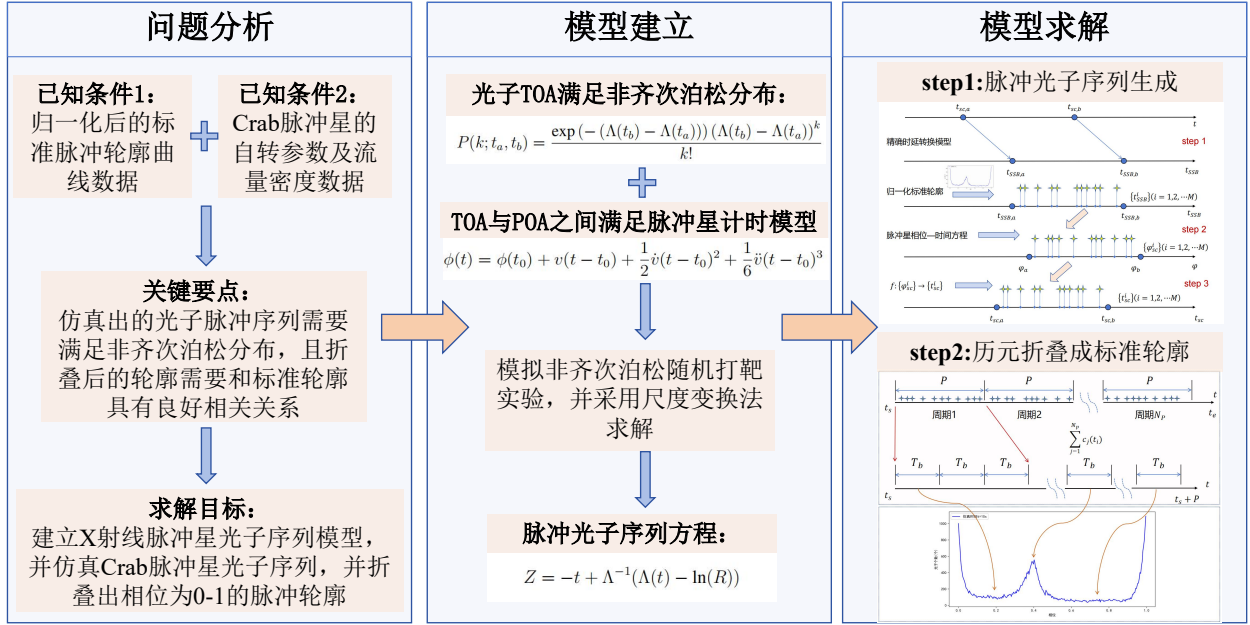


图 6.1 问题四流程图

由于脉冲光子的 TOA 非齐次泊松分布，并且需要考虑脉冲星自行以及时间系统转换等方面，因此本文采用非齐次泊松方程，脉冲星计时模型，逆变换采样及脉冲星光子序列方程等函数模型构建脉冲星的仿真模型，实现对脉冲星光子序列的精确仿真，并借助尺度变换法将其折叠至一个周期，并与标准轮廓对比，通过多个评估方法与指标对仿真的脉冲光子序列进行评估。

6.1 问题分析

在脉冲星深空导航与授时领域，脉冲星信号的研究对于未来航天器的自主导航有重要意义。脉冲星，尤其是 X 射线脉冲星，如 Crab 脉冲星，其稳定的周期信号可以用于航天器的位置和时间标定。然而，X 射线探测器的发射成本高昂，实际的探测数据获取难度较大。在缺乏实测数据的情况下，仿真脉冲星信号是研究脉冲星导航的有效手段。通过仿真手段同样可以帮助更好地理解脉冲星信号的辐射过程。

本题目要求在给定的仿真条件下，建立 X 射线脉冲星光子序列模型，并仿真 Crab 脉冲星光子序列，在获取了仿真时间内的光子序列后，将这些光子折叠成一个脉冲轮廓。折叠过程中需根据 Crab 脉冲星的自转参数，将不同时间段的光子到达时间归一到一个周期

内，从而得到整个脉冲的轮廓。

6.2 模型建立

6.2.1 脉冲星光子序列仿真

由于脉冲星位于遥远的太阳系外，其辐射的 X 射线信号需经过长时间才能到达探测器，因此探测器所探测到的为离散的光子 TOA 序列，可认为这些序列脉冲星光子到达探测器的时间服从非齐次泊松分布

$$P(k; t_a, t_b) = \frac{\exp(-(\Lambda(t_b) - \Lambda(t_a))) (\Lambda(t_b) - \Lambda(t_a))^k}{k!} \quad (6.1)$$

其中，累计函数 Λ 可表示为 $\Lambda = \int_0^t \lambda(s) ds$ ， λ 为速率函数（即脉冲星的流量密度函数），可表示为 $\lambda(t) = \lambda(\phi(t)) = \lambda_b + \lambda_s h(\phi(t))$ ，其中 λ_b 、 λ_s 分别为背景光子流量（背景噪声）与脉冲星的流量， h 为归一化的脉冲轮廓函数，而脉冲星计时模型 ϕ 则用来表示相位与时间的关系，用来预报脉冲星的自转相位和脉冲到达太阳系质心的时间。对于毫秒脉冲星而言，周期变化率小，略去频率二阶导数以上的高阶项，脉冲星计时模型可以表示为：

$$\phi(t) = \phi(t_0) + v(t - t_0) + \frac{1}{2} \dot{v}(t - t_0)^2 + \frac{1}{6} \ddot{v}(t - t_0)^3 \quad (6.2)$$

为获得航天器处的光子 TOA 序列，需对时间转化模型进行逆向求解。接下来将介绍非齐次泊松分布数值的模拟方法及利用时间转换模型逆向求解光子 TOA 序列的方法。本研究选用反函数法模拟非齐次泊松分布数值，首先定义随机变量 X ：

$$X = F^{-1}(R) \quad (6.3)$$

其中， R 为在 $[0,1]$ 上均匀分布的随机变量， F 为其分布函数， F^{-1} 为分布函数 F 的反函数，由此可得：

$$F_X(n) = P\{X \leq n\} = P\{F^{-1}(R) \leq n\} \quad (6.4)$$

因为分布函数是单调函数，所以当且仅当 $R \leq F(n)$ 时，满足 $F^{-1}(R) \leq n$ 。因此，从上式可得到

$$F_X(a) = P(R \leq F(n)) = F(n) \quad (6.5)$$

因此通过分布函数 F ，计算反函数 F^{-1} ，产生随机变量 R ，计算 $X = F^{-1}(R)$ ，可获得分布函数 F 的随机变量 X 。

由于光子到达时间服从非齐次泊松分布，在 $t_n = t$ 时刻探测到第 n 个光子到达的条件下，假设经过时间间隔 Z ，探测到下一个光子，则 Z 大于 z 的概率可表示为因此通过分布函数 F 及其反函数 F^{-1} ，并生成一组随机变量 R ，计算 $X = F^{-1}(R)$ 来获取分布函数相应的随机变量。

对于脉冲星的光子到达时间，可认为其遵循非齐次泊松分布，建设 $t_n = t$ 时刻探测到第 n 个光子到达的条件下，探测到下一个光子的时间间隔为 Z ，那么 Z 大于 z 的概率可用相应的公式来表示，具体形式为

$$F_X(a) = P(R \leq F(n)) = F(n) \quad (6.6)$$

$$\begin{aligned} P(Z > z | t_n = t) &= P(N_{t+z} - N_t = 0) \\ &= \exp\left(-\int_t^{t+z} \lambda(t) dt\right) \\ &= \exp(-(\Lambda(t+z) - \Lambda(t))) \end{aligned} \quad (6.7)$$

$$P(Z \leq z | t_n = t) = 1 - F_{Z|t_n=t}(z | t_n = t) \quad (6.8)$$

式中 N_t 为光子总个数， $\lambda(t)$ 是光子到达的速率函数， $\Lambda(t)$ 是积分速率函数。 $F_{Z|t_n=t}(z | t_n = t)$ 表示在 $t_n = t$ 时刻探测到一个光子下， Z 小于 z 的分布函数。

联合式 (6.7) 和式 (6.8) 可得

$$F_{Z|t_n=t}(z | t_n = t) = 1 - e^{-(\Lambda(t+z) - \Lambda(t))} \quad (6.9)$$

则 $F_{Z|t_n=t}(z | t_n = t)$ 的反函数为

$$F_{Z|t_n=t}^{-1}(z | t_n = t) = t + \Lambda^{-1}(\Lambda(t) - \ln(1 - z)) \quad (6.10)$$

则在 $t_n = t$ 时刻探测到光子的条件下，时间间隔 Z 为

$$Z = -t + \Lambda^{-1}(\Lambda(t) - \ln(1 - R)) \quad (6.11)$$

式中 R 是区间 $[0,1]$ 之间的均匀随机变量，该式也可表示为

$$Z = -t + \Lambda^{-1}(\Lambda(t) - \ln(R)) \quad (6.12)$$

由于脉冲轮廓通常不规则， $\Lambda(t)$ 和 Λ^{-1} 很难解析求出。因此采用近似模型来简化计算，若相邻光子到达的时间间隔很小，即 $\delta t \rightarrow 0$ 时，可认为

$$t' \approx t - \frac{\ln(R)}{\lambda(t)} \quad (6.13)$$

本模型为模拟光子到达时间提供了一种有效手段，至此，已实现在 SSB 处模拟光子的 TOA 序列，而获航天器处的光子的 TOA 序列可参照图6.2。

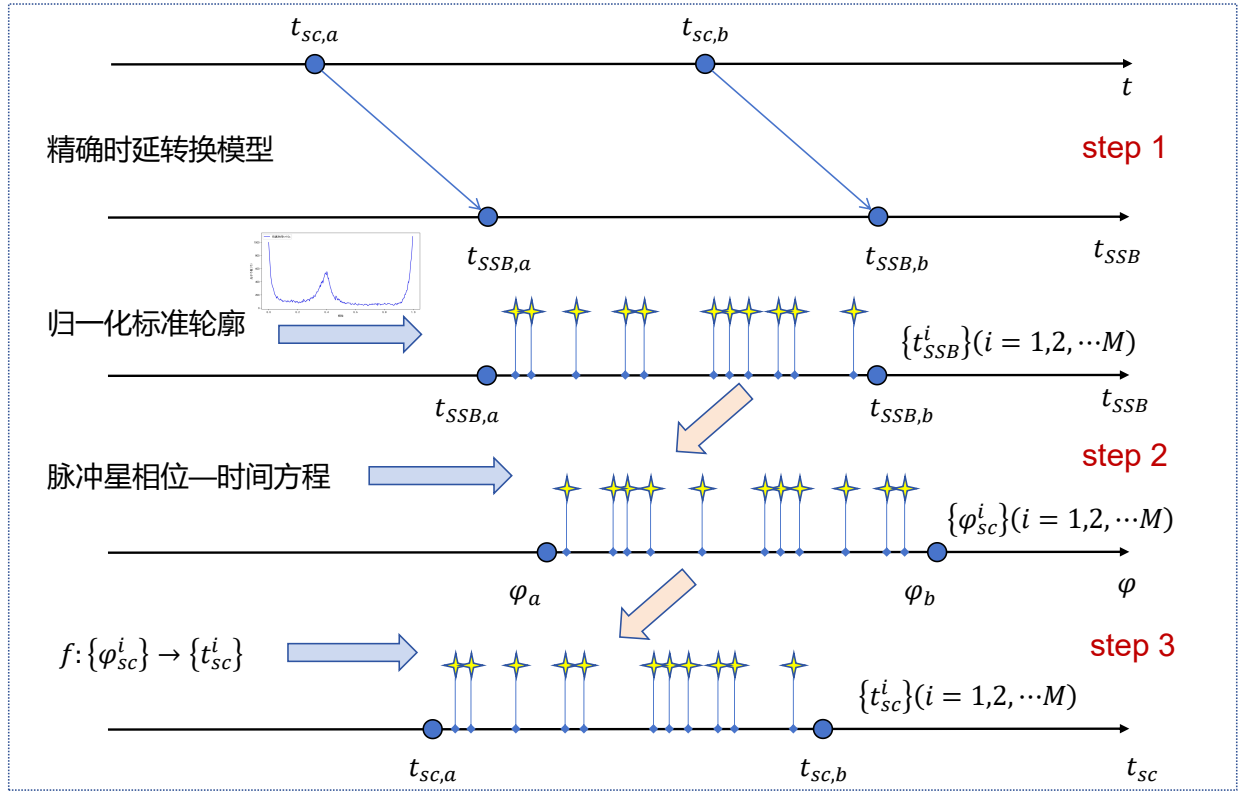


图 6.2 脉冲光子序列生成

此过程具体采用以下步骤:

(1) 将航天器处的模拟时段转换到 SSB 处

首先根据航天器在不同时刻的状态信息, 将航天器所处时间段 $(t_{sc,a}, t_{sc,b})$ 转换至太阳系参考质心 SSB 处所对应的模拟时段 $(t_{SSB,a}, t_{SSB,b})$ 。

(2) SSB 处光子 POA 序列的模拟

通过模拟脉冲星的标准轮廓与其自转相位模型, 采用数值模拟法生成 SSB 处的光子到达时间 TOA 序列 $\{t_{SSB}^i | i = 1, 2, \dots, M\}$, 并在此基础上, 结合光子信号的相位模型, 计算相应的 POA 序列 $\{\phi_{SSB}^i | i = 1, 2, \dots, M\}$ 。由于光子到达航天器处的相位与到达 SSB 处光子相位相等, 满足 $\phi_{SSB}^i = \phi_{sc}^i (i = 1, 2, \dots, M)$, 因此光子在航天器处的相位序列即为航天器处的光子 POA 序列 $\phi_{sc}^i (i = 1, 2, \dots, M)$ 。

(3) 航天器处光子 POA 与光子 TOA 的转换

通过已知的脉冲星频率参数及航天器的运动状态信息, 获取航天器处的脉冲星频率参数, 并建立航天器处光子 POA 序列和光子 TOA 序列转换模型。可由光子 POA 序列 $\{\phi_{sc}^i\}$ 计算航天器处的光子 TOA 序列 $\{t_{sc}^i\}$ 的转换过程, 若可推导出两者的转换关系式, 即可完成光子 POA 到光子 TOA 的直接转换, 过程将得到简化。已知 SSB 处的相位模型, 同样可推算航天器处的在某时刻所接收到的脉冲星 X 射线光子信号的相位, 具体公式如下式:

$$\phi_{sc}(t) = \phi(t + \tau(t)) \quad (6.14)$$

其中, $\tau(t)$ 为时间转换项, 包括脉冲星光子至太阳系质心 SSB 的传播时间项以及航天器内的固有时间项。通过上式同样可实现 SSB 处模拟的光子 TOA 序列至航天器处的光子 TOA 序列的转换。

6.2.2 脉冲星轮廓构建及检验

脉冲星的周期可通过采用周期搜索的方式来确定, 然后采用历元折叠方法获得脉冲星的模板轮廓。图6.3为历元折叠获取脉冲轮廓的示意图, 其核心思想是: 假设探测器观测脉冲星的时间为 T_{obs} , 在此时间段内包含了 N 个脉冲星周期 P , 即有 $T_{\text{obs}} \approx N \cdot P$ 。每个周期 P 可以划分为 N_b 个时间区间 (称为 bin 块), 每个 bin 的长度为 T_b 。通过将探测器探测到的光子到达时刻折叠回第一个周期, 并统计每个 bin 块中的光子数量。之后对各 bin 块中的光子数量进行归一化处理, 生成相应的脉冲轮廓。那么在第 i 个 bin 中, $i \in [1, N_b]$, 归一化后的轮廓 $\tilde{\lambda}(T_i)$ 表示为:

$$\tilde{\lambda}(T_i) = \frac{1}{NT_b} \sum_{j=1}^N c_j(T_i) \quad (6.15)$$

其中, $c_j(T_i)$ 为第 j 个周期的第 i 个 bin 块内的光子数, T_i 为第 i 个 bin 块的中心对应的时刻。

为了验证模拟脉冲星光子信号是否服从泊松分布, 可采用 χ^2 拟合优度检验法, 可以选取某一时间段内的 X 射线光子数 $a = \{0, 1, 2, \dots, a_i\}, i = 1, 2, 3, \dots$ 作为样本, 记样本中等于 a_i 的个数为 n_i , 并设 a 共有 k 个不同的取值, 则 $a = a_i = i - 1$ 的概率 p_i 为:

$$p_i = P\{a = i - 1\} = \frac{E(N_s^t)^{i-1} \exp(-E(N_s^t))}{(i-1)!} \quad (6.16)$$

其中 E 在本问题中代表观测时刻的平均光子数目, 其具体表达公式如下

$$E(N_s^t) = \int_t^s \lambda(\xi) d\xi = N \int_0^T \lambda(\xi) d\xi \quad (6.17)$$

其中 N 代表模拟的周期个数, 样本总数为 $n = \sum_{i=1}^k n_i$, 则皮尔逊 χ^2 统计量为

$$\chi^2 = \sum_{i=1}^k \frac{(n_i - np_i)^2}{np_i} \quad (6.18)$$

在给定的显著性水平 α 下, 如果 $\chi^2 \geq \chi_{\alpha}^2(k-1)$ 时, 则认为模拟的信号不服从泊松分布, 反之, 可认为其遵循泊松分布。

此外, 可对所构建出的轮廓采用 Pearson 系数计算其与标准轮廓的重叠度, 以评估仿真 Pearson 系数计算公式如下:

$$R_{\text{sp}} = \frac{\sum_{k=1}^N (s_k - \bar{s})(w_k - \bar{w})}{\sqrt{\sum_{k=1}^N (s_k - \bar{s})^2} \sqrt{\sum_{k=1}^N (w_k - \bar{w})^2}} \quad (6.19)$$

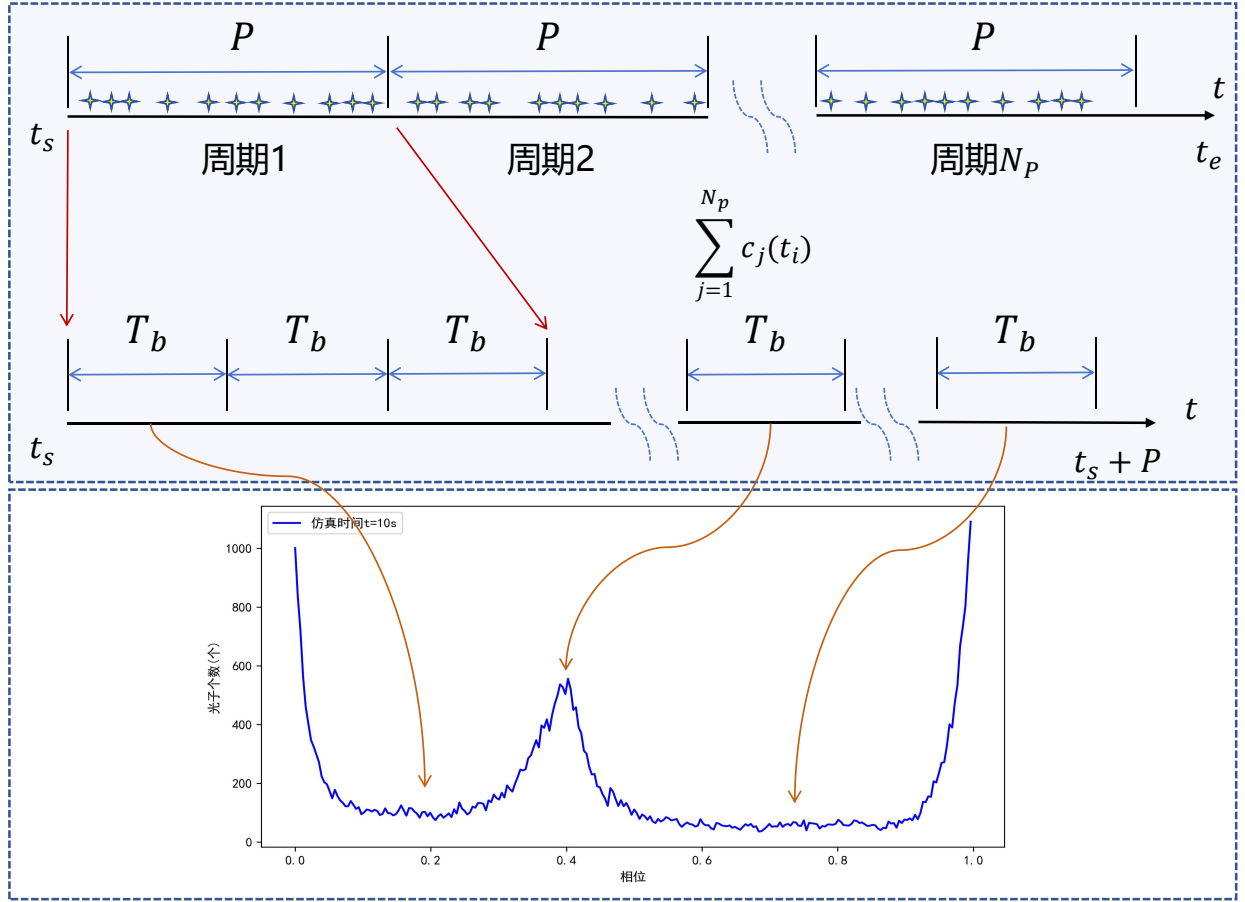


图 6.3 历元折叠

其中， s_k 为模拟脉冲轮廓， w_k 为标准脉冲轮廓。

6.2.3 仿真精度提高

基于上述信号仿真方法，在仿真过程时，存在一些可改进之处，例如，当某些情况下脉冲星光子流量较小时，此时 $\delta t \rightarrow 0$ 的条件无法实现，此时不能采用式 (6.13) 进行脉冲星光子信号模拟。因此，本研究针对此处进行了改进，采用更高效而非齐次泊松过程的模拟方法即逆变换采样法，其详细原理如下：

根据式 (6.12)，设第 n 个光子到达时间为 t_n ，则第 $n+1$ 个光子到达时间可表示为：

$$t_{n+1} = \Lambda^{-1}(\Lambda(t_n) + R) \quad (6.20)$$

可对上式进行处理，设 $\tau_i = A(t_i) + R$ ，当式 (6.20) 成立时， $\tau_i (i = 1, 2, \dots)$ 是一个参数为 1 的齐次泊松过程的 TOA 序列，其证明过程如下：

$$\begin{aligned}
\tau_{i+1} - \tau_i &= (\Lambda(t_{i+1}) + R) - (\Lambda(t_i) + R) \\
&= \Lambda(t_{i+1}) - \Lambda(t_i) \\
&= \Lambda(\Lambda^{-1}(\Lambda(t_i) + R)) - \Lambda(t_i) \\
&= \Lambda(t_i) + R - \Lambda(t_i) \\
&= R.
\end{aligned} \tag{6.21}$$

上式表明 $\tau_{i+1} - \tau_i$ 服从参数为 1 的指数分布，从而证明了前文所提到的 τ_i 服从参数为 1 的齐次泊松过程。因此式 (6.20) 可以改写为：

$$t_i = \Lambda^{-1}(\tau_i) \tag{6.22}$$

其中 t_i 为积分速率函数 Λ 的非齐次泊松过程的到达时间。

为了模拟积分速率函数为 Λ 的非齐次泊松过程光子到达时间 TOA 序列 $t_i (i = 1, 2, \dots)$ ，首先需生成一系列参数为 1 的齐次泊松过程序列 $\tau_i (i = 1, 2, \dots)$ ，然后对这些序列进行逆变换采样。由于齐次泊松过程在单位时间内的到达数服从参数为 1 的泊松分布，并在此基础上，根据泊松分布的性质，可认为其到达时间间隔服从 $R(0, 1)$ 变量。根据式 (6.20) 对集成速率 Λ 进行仿真，特别的是，此过程不需实时解算，只需将 $\tau_i (i = 1, 2, \dots)$ 由式 (6.22) 映射至非齐次泊松过程的到达时间序列，接下来为此过程的详细流程介绍：

1) 计算航天器处的离散积分速率函数： $\Lambda(t^k) = \sum_{i=1}^k \lambda_{sc}(t^i) t_{\text{sample}} \quad (k = 1, 2, \dots, K)$ ，满足 $t_s < t^i < t^{i+1} \leq t_e$ ，且 $t^{i+1} - t^i = t_{\text{sample}}$ ，并且 $K t_{\text{sample}} = t_e - t_s$ ，其中 t_{sample} 为采样间隔，并令 $t_0 = t_s$ ，且 $\Lambda(t^0) = 0$ 。一般情况下取 t_{sample} 小于或等于脉冲星标准脉冲轮廓的一个 bin 块时长。 $\Lambda(t^k)$ 即为观测时段 $[t_s, t_e]$ 内到达航天器的平均光子个数。

2) 根据泊松分布产生一个参数为 $\Lambda(t^K)$ 的泊松随机数 I ，并认为其为观测时段内实际到达的光子个数。

3) 利用所得到的泊松随机数 I 生成一系列随机变量 $e_i (i = 1, 2, \dots, I)$ ，进而生成齐次泊松分布的时间序列 $\tau_i = \Lambda(t^k) e_i (i = 1, 2, \dots, I)$ 。

4) 通过式 (6.22) 对 $\tau_i (i = 1, 2, \dots, I)$ 进行逆变换采样求得非齐次泊松过程到达时间 $t_i (i = 1, 2, \dots, I)$ 。此过程可通过对积分速率函数进行线性插值来实现，插值公式为 $t_i = t^{k-1} + \frac{\tau_i - \Lambda(t^{k-1})}{\Lambda(t^k) - \Lambda(t^{k-1})} t_{\text{sample}} \quad [\Lambda(t^{k-1}), \Lambda(t^k)]$ 为 τ_i 所处的时间范围。

5) 将所生成的时间序列按从小到大进行排列，即可得到航天器在观测时段 $[t_s, t_e]$ 内的光子 TOA 序列。

通过以上步骤，整个模拟过程可以简化为数组操作，而无需要采用递推的流程；生成随机数即为光子个数，而常用的齐次泊松过程筛选法需要产生远大于光子数的随机数，然后再利用筛选出非齐次泊松过程的到达时间，因此使用本方法可显著提高效率，节省时间。

6.3 模型求解与结果分析

(1) 计算脉冲星自转相位

根据式 (6.2)，利用脉冲星自转频率计算给定时刻的相位，此相位是生成脉冲星光子到达时刻 TOA 的核心，因为光子流量密度依赖于脉冲星自转相位。

(2) 计算光子流量密度

脉冲星光子流量密度 $\lambda(t)$ 为背影光子星流量 λ_b 和脉冲星信号流量 $\lambda_s h(\phi(t))$ 的总和，其中 $h(\phi(t))$ 是脉冲星的归一化脉冲轮廓函数。该函数可通过多种方式求得：

1. 线性插值方法原理：通过找到与给定的相位 $\phi(t)$ 最接近的两个相位点 ϕ_1 和 ϕ_2 并计算这两个点。利用线性插值公式计算 $h(\phi(t))$ ，即 $h(\phi(t)) = h_1 + \frac{h_2 - h_1}{\phi_2 - \phi_1} \cdot (\phi(t) - \phi_1)$ ，从而实现脉冲星的归一化脉冲轮廓函数 $(\phi(t))$ 的获取。
2. 分段指数拟合方法原理：先将所提供的标准轮廓划分为多个区间（如 0 到 0.2 为一个区间，0.2 到 0.4 为一个区间），采用不同的指数分段拟合标准轮廓的不同部分，例如 $h(\phi(t)) = a \cdot e^{b \cdot \phi(t)} + c$ ，最后找到合适的参数 a , b , c ，使得拟合的函数尽可能贴合标准轮廓函数，即可得到给定相位下的脉冲星的归一化脉冲轮廓函数 $h(\phi(t))$ 。
3. 三次样条插值方法原理：通过构建一个分段的三次多项式来插值数据，其形式可表示为：

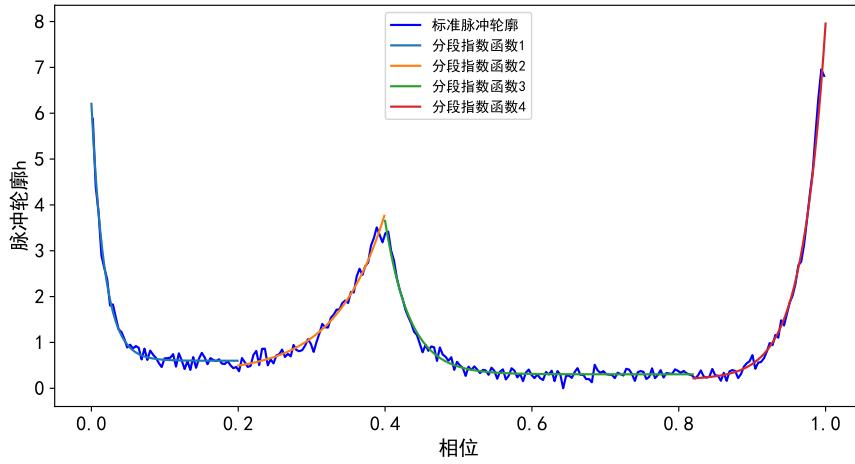
$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3, \quad i = 0, 1, \dots, n-1 \quad (6.23)$$

每个相邻的数据点都用一段三次多项式来连接，要求这些多项式的导数和其二阶导数在每个分段的边界连续即

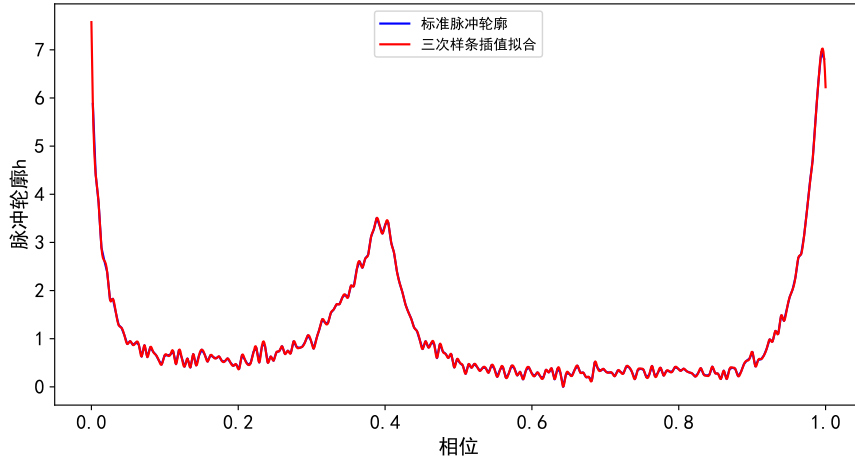
$$S_i(x_{i+1}) = S_{i+1}(x_{i+1}), \quad S'_i(x_{i+1}) = S'_{i+1}(x_{i+1}), \quad S''_i(x_{i+1}) = S''_{i+1}(x_{i+1}) \quad (6.24)$$

进而保证插值函数的平滑性，获得准确可靠的归一化脉冲轮廓函数 $h(\phi(t))$ 。

图6.4给出了利用分段指数拟合和三次样条插值的结果。图中分段线性函数拟合的结果整体较为平滑，但在分段处函数值不连续；三次样插值的拟合结果基本和标准轮廓重合，拟合精度较高。



(a) 分段指数函数拟合



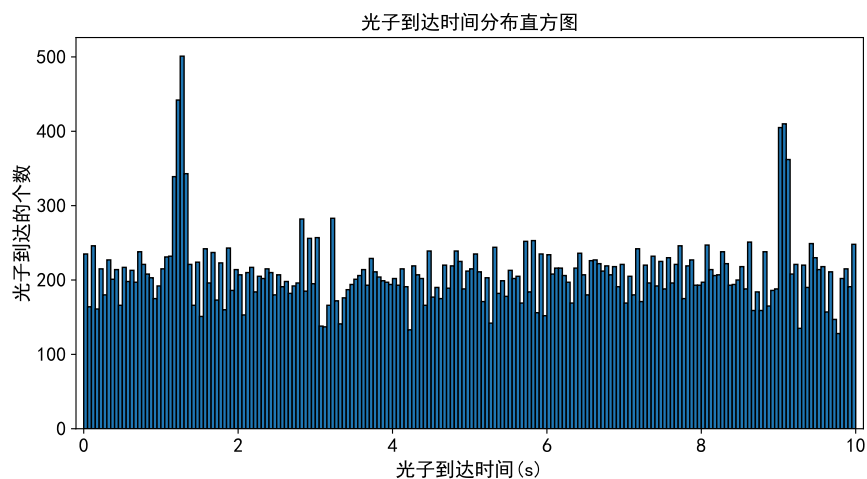
(b) 三次样条插值

图 6.4 Crab 脉冲星的标准脉冲轮廓曲线拟合

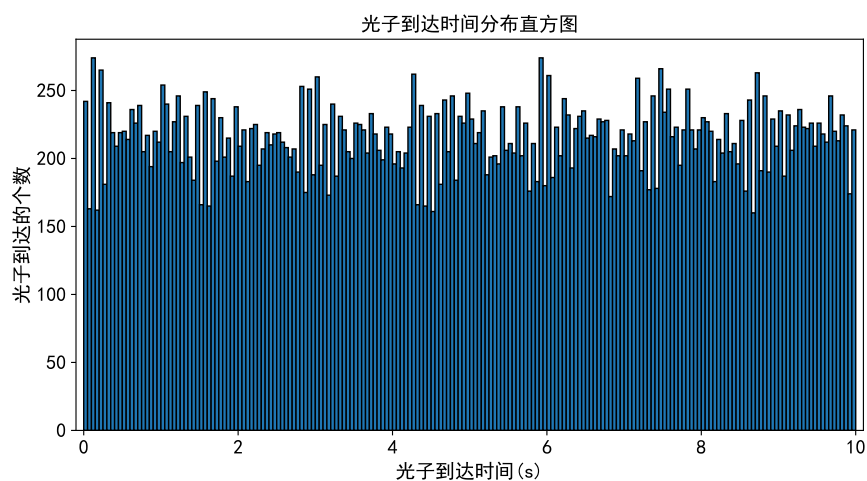
(3) 生成光子到达时间序列

使用上述过程计算的脉冲星光子密度函数 $\lambda(t)$ ，按照前文所提到的脉冲星时间序列的近似递推模型（式 (6.13)），可以依次计算下一个光子到达时间，进而递推生成光子到达时间序列。

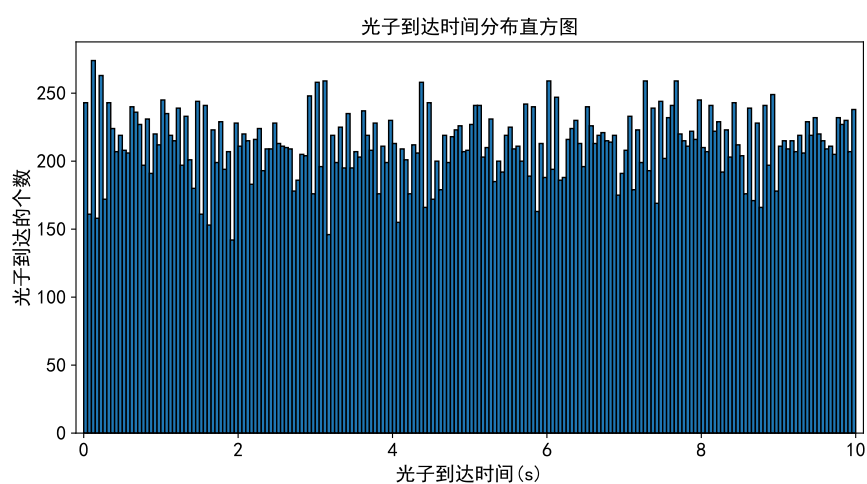
基于不同插值函数方法可以得到 Crab 脉冲星光子序列（时间分布直方图）。如图6.5所示，X 轴表示仿真时间，被均分为 200 个区间，Y 轴表示对应区间内到达的光子数。由图可知，基于分段指数拟合和三次样条插值得到的光子时间序列整体上分布较为均匀，每个时间区间平均到达的光子数在 200 个左右，没有很明显的跳点，符合实际情况。而基于线性插值得到的光子时间有两个较为明显的凸起，这有可能是因为线性插值得到的结果在某些区间内没有很好的反映出标准脉冲轮廓的变化趋势。



(a) 线性插值



(b) 分段指数拟合



(c) 三次样条插值

图 6.5 基于不同插值方法得到的 Crab 脉冲星光子序列

(4) 折叠光子序列得到脉冲轮廓

通过历元折叠法，而在采用历元折叠法时，子相位间隔数 N 的取值也是一个需要研究的问题。此时可计算不同 N 值时的脉冲轮廓信噪比，其计算公式如下：

$$\text{SNR} = \frac{N_{\text{peak}}}{\sqrt{N_{\text{total}}}} \sqrt{p} \quad (6.25)$$

其中， N_{peak} 为脉冲轮廓主峰的光子数， N_{total} 为观测光子总数， p 为曝光时间。

根据文献所描述，在进行历元折叠时通常选取 256 作为 N 值，在实际求解过程中，尝试对 bin 值做适当的调整调整，发现若 N 取值太大，则折叠后的脉冲轮廓会损失信噪比；若 N 取值过小，折叠后的轮廓将丢失脉冲信号的频域信息。

因而，本研究选取的子相位间隔数为 256，将光子到达时间序列归算至一个周期内并统计每个时间区间（ bin ）中的光子个数，得到脉冲星的轮廓。

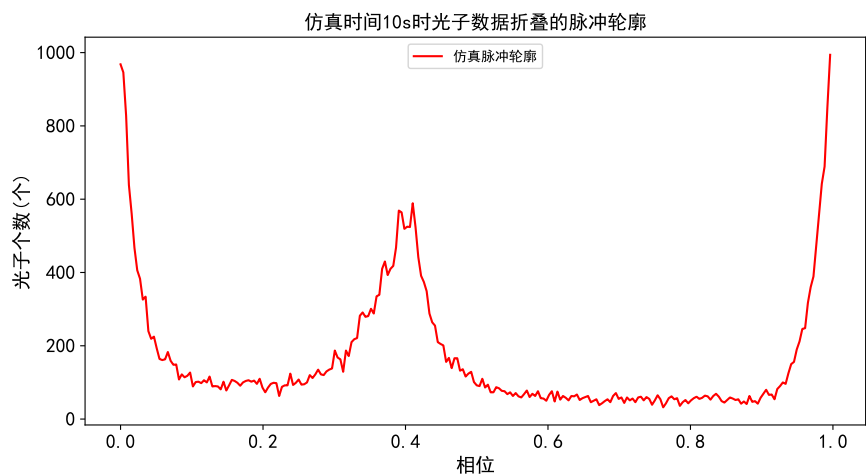
对上一步得到的脉冲星光子序列进行折叠，可以得到如图6.6所示的仿真脉冲轮廓。图中，三个脉冲轮廓变化趋势较为一致，说明本文6.2.1节提出的 Crab 脉冲星光子序列生成模型能够很好的进行脉冲星光子序列的模拟。

(5) 仿真脉冲轮廓评估

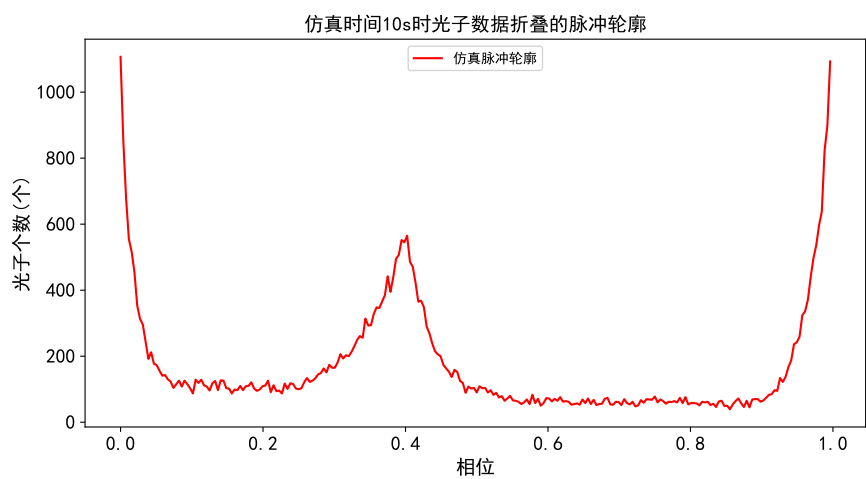
为了对仿真结果进行评估可验证模拟生成的脉冲信号是否遵循泊松分布，本研究实际评估时先采用 Pearson 系数计算其与标准轮廓的重叠度，计算公式同式 (6.19)，然后选取 Pearson 系数最大的模拟方法进行 χ^2 拟合优度检验法，检查模拟的光子信号是否服从泊松分布，对应的皮尔逊 χ^2 统计量同式 (6.18)。通过以上两个指标实现对仿真模型以及仿真结果的双重检验，以保证仿真的准确性和可靠性。

将上一步得到的仿真脉冲轮廓与标准脉冲轮廓进行对齐，可得到如图6.7所示的结果。根据图6.7，可以得到这三个仿真脉冲轮廓与标准脉冲轮廓的 Pearson 系数，如表6.1所示，可以看到，三次样条插值得到的 Pearson 系数比其余两者大，这表明基于6.2.1节提出脉冲光子时间序列模型，使用三次样条插值得到的脉冲轮廓最接近标准轮廓。因此后续对使用三次样条插值的生成脉冲光子序列的模型进行 χ^2 拟合优度检验。

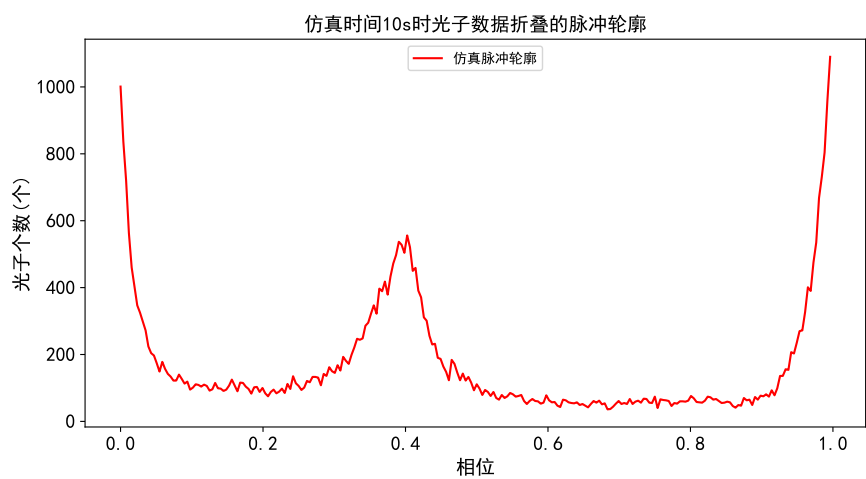
本次研究生成 30 个脉冲周期，约 1s 的观测数据进行模拟测试。根据6.2.2节，可知 $E(N_s^t) = 4283$ ，则本次可模拟生成 $2E(N_s^t)$ 个样本数据， n_i 的分布如图6.8所示，由图可见，在光子数集中在期望值附近，而大于或小于期望达到某一值后，样本数量变为 0。将 n_i 代入式6.18求得 $\chi^2=24.98915$ 。通过模拟的样本可知 $k = 418$ ，则 $\chi_{1-\alpha}^2(k-1) = 465.61139$ ，故 $\chi^2 < \chi_{1-\alpha}^2(k-1)$ ，说明本研究模拟产生的光子到达时间序列服从泊松分布。



(a) 线性插值

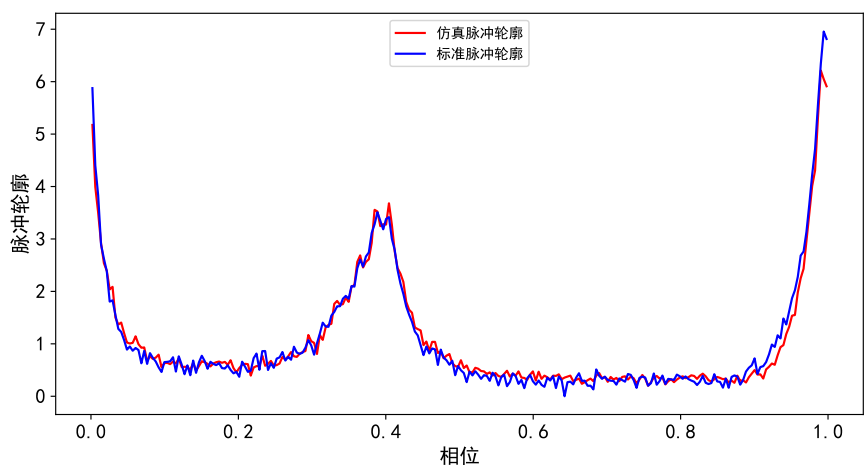


(b) 分段指数拟合

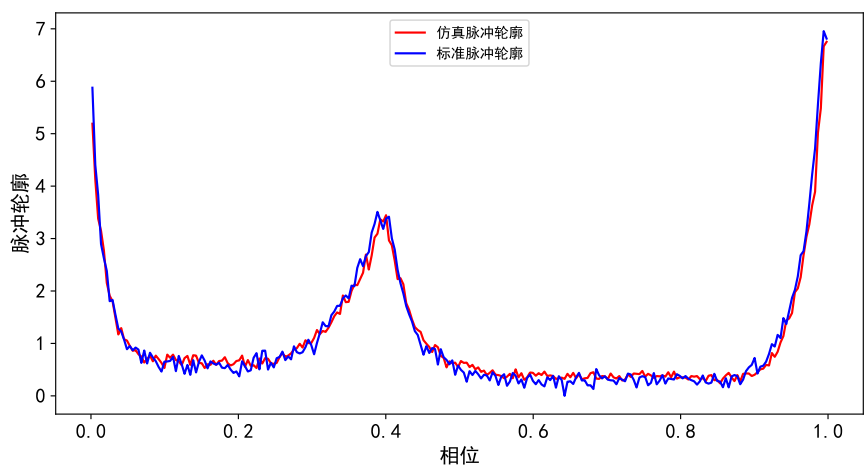


(c) 三次样条插值

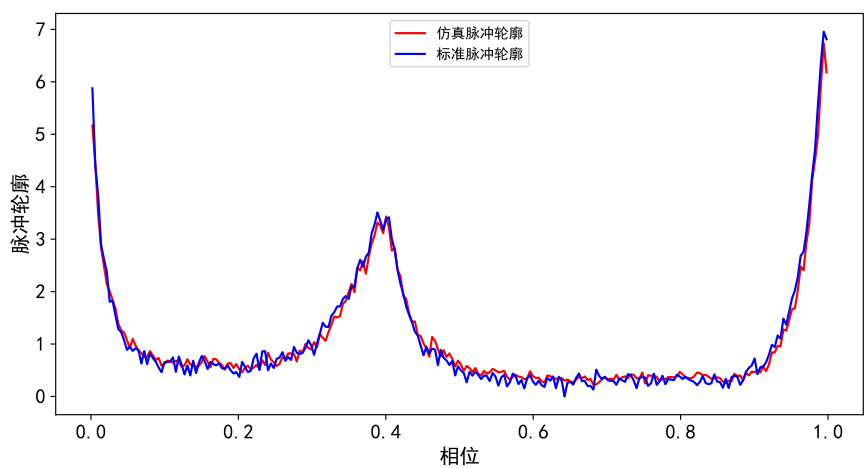
图 6.6 基于不同插值方法得到的仿真脉冲轮廓



(a) 线性插值



(b) 分段指数拟合



(c) 三次样条插值

图 6.7 不同插值方法得到的仿真脉冲轮廓与标准脉冲轮廓对比

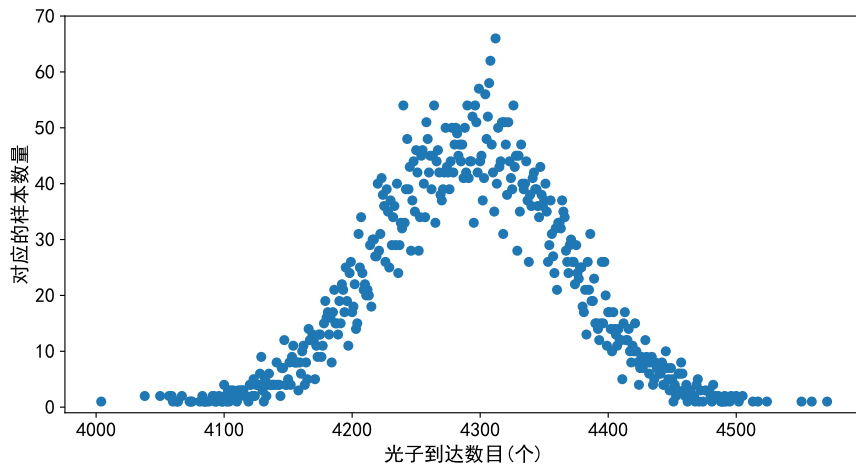


图 6.8 模拟样本的光子数目分布图

表 6.1 基于6.2.1节模拟脉冲序列生成方法采用不同插值方式得到的 Pearson 系数

插值方式	Pearson 系数
(a) 线性插值	0.99042
(b) 分段指数拟合	0.99089
(c) 三次样条插值	0.99351

(6) 提高仿真精度

在某些情况下，脉冲星光子流量较小，并不满足6.2.1节提出的模拟脉冲光子模型中假设条件 $\delta t \rightarrow 0$ 。针对这个问题，本研究使用另外一种基于逆变换采样法的高效非齐次泊松过程模拟方法，该方法不采用递推的方式生成光子时间序列，而是根据齐次泊松分布的时间序列通过逆变换的方法来获取光子的非齐次泊松分布的时间序列。

图6.9、图6.10分别给出了在保持上述背景光子流量密度值不变基础上，采用逆变换采样法得到的 Crab 脉冲星光子序列以及得到的仿真脉冲轮廓，整体上来看，与6.2.1节生成仿真脉冲时间序列的模型具有较好的一致性。

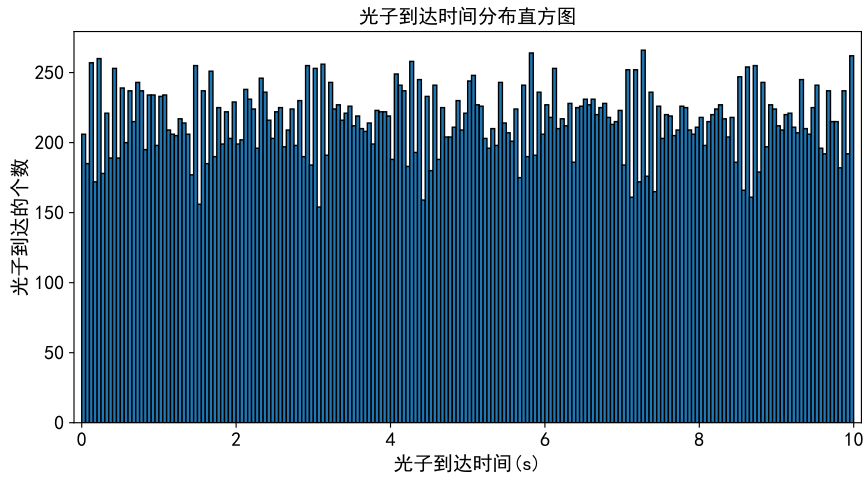


图 6.9 采用逆变换采样法得到的 Crab 脉冲星光子序列

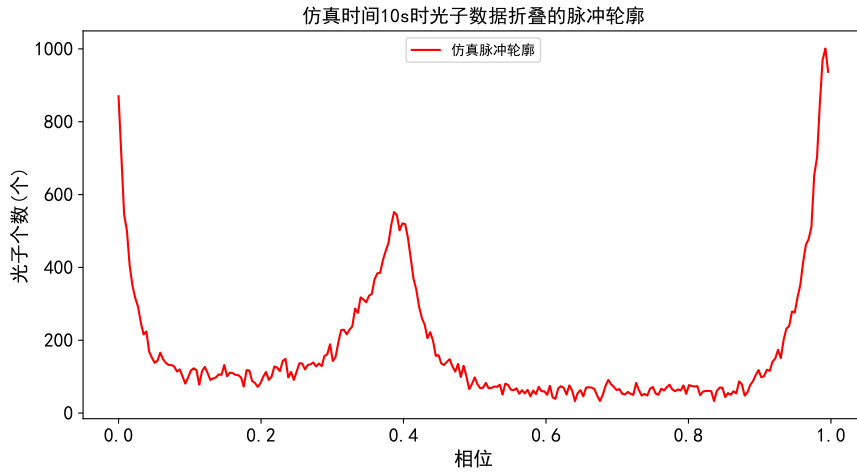


图 6.10 采用逆变换采样法得到的仿真脉冲轮廓

为了进一步对比6.2.1节仿真方法与逆变换采样法之间的差异，本研究将图6.10得到的模拟脉冲轮廓与标准脉冲脉冲进行对齐，得到结果如图6.11所示，然后得到逆变换采样法的 Pearson 系数为 0.99351，优于表6.1中的 Pearson 系数。这表明逆变换采样法模拟出来的脉冲光子轮廓与标准脉冲轮廓一致性更好，仿真精度更高，可以更好地展现脉冲星的辐射特性。

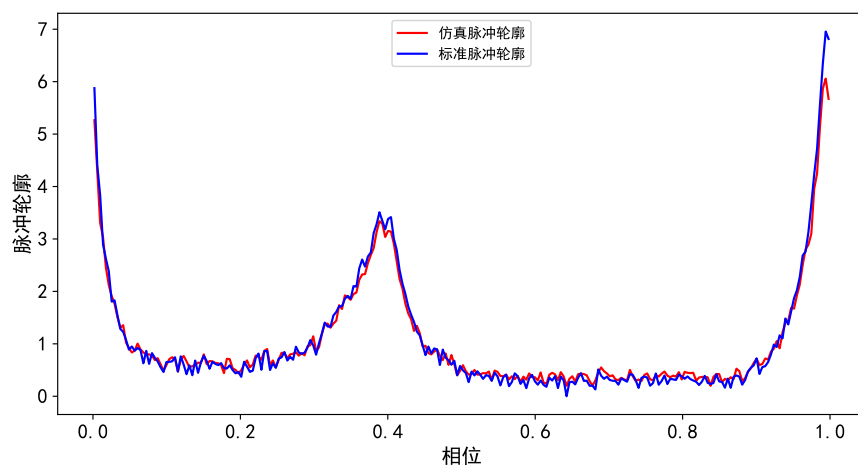


图 6.11 采用逆变换采样法得到仿真脉冲轮廓与标准脉冲轮廓对比

7 模型评价

本文建立了从卫星到太阳之间的脉冲光子时延模型时，模型不仅考虑了几何传播时延，还综合引入了 Shapiro 时延、引力红移时延和狭义相对论的动钟效应，确保了时延计算的全面性。另外，本文建立了脉冲光子仿真时间序列模型，成功仿真出了 Crab 脉冲星光子到达的时间序列，并且仿真的脉冲轮廓结果与标准脉冲轮廓高度一致，两者之间相关性最高达到了 0.99351，这证明了本文所使用的模型具有很高的仿真精度。此外，本文利用 χ^2 拟合优度对使用的脉冲模型进行了检验，检验结果表明本文模拟的脉冲光子时间序列很好地服从泊松分布，确保模型具有良好的适用性和稳健性。另外，考虑到实际脉冲光子的流量有时是比较少的，不满足两个光子到达时间间隔趋近于 0，本文提出了一种逆变换采样法来进行建模，结果表明基于逆变换采样法得到的仿真脉冲轮廓与标准脉冲轮廓的一致性更好，可以更好地展现脉冲星的辐射特性。

在建立脉冲光子时延模型时，本文只考虑了与太阳相关的时延误差项，而忽略了太阳系中其他天体对时延的影响，这会影响卫星和太阳之间高精度的时延确定。另外，在模拟脉冲光子时间序列时，本文所采用的模型认为短时间内脉冲星自转频率的是较为稳定的，但是如果脉冲星自转频率发生一些较大的变化或者脉冲观测时刻到脉冲参考时刻跨度过大时，模型仿真得到的脉冲光子时间序列可能会和标准脉冲轮廓产生较大的误差。

参考文献

- [1] 郑伟,王禹淞,姜坤,等.X射线脉冲星导航方法研究综述[J].航空学报,2023,44(03):26-42.
- [2] 黎胜亮,刘昆,吴锦杰.脉冲轮廓建模仿真与去噪[J].国防科技大学学报,2013,35(04):26-29+34.
- [3] 苏剑宇.深空探测X射线脉冲星导航方法研究[D].西安电子科技大学,2023.
- [4] 桂先洲,黄森林,孙晨.重构X射线脉冲星信号的纯数值模拟新算法[J].国防科技大学学报,2015,37(02):143-148.
- [5] 易韦韦,偶晓娟,许静文,等.脉冲星导航试验卫星观测数据处理与分析[J].深空探测学报,2018,5(03):241-245+261.
- [6] 薛梦凡,李小平,孙海峰,等.一种新的X射线脉冲星信号模拟方法[J].物理学报,2015,64(21):487-497.
- [7] 王浙宇,韩孟纳,童明雷.XPNAV-1和NICER对Crab脉冲星观测数据的比较分析[J].时间频率学报,2023,46(03):178-187.
- [8] 李建勋.基于X射线脉冲星的定时与自主定位理论研究[D].西安理工大学,2009.
- [9] 黄良伟,帅平,张新源,等.脉冲星导航试验卫星时间数据分析与脉冲轮廓恢复[J].中国空间科学技术,2017,37(03):1-10.
- [10] [1] 杨廷高,童明雷,赵成仕,等.Crab脉冲星X射线计时观测数据处理与分析[J].天文学报,2018,59(02):10-16.
- [11] Edwards R T, Hobbs G B, Manchester R N. TEMPO2, a new pulsar timing package—II. The timing model and precision estimates[J]. Monthly Notices of the Royal Astronomical Society, 2006, 372(4): 1549-1574.
- [12] Curtis H D. Orbital Mechanics for Engineering Students, Burlington: Elsevier, 2005.
- [13] Emadzadeh A A, Speyer J L. Navigation in Space by X-ray Pulsars, New York: Springer, 2011.
- [14] 苏哲,许录平,甘伟.基于压缩感知的脉冲星轮廓构建算法[J].中国科学:物理学力学天文学,2011,41(05):681-684.
- [15] 胡慧君,赵宝升,盛立志,等.用于脉冲星导航的X射线光子计数探测器研究[J].物理学报,2012,61(01):526-531.
- [16] 童明雷,杨廷高,赵成仕,等.脉冲星计时模型参数的测量精度分析与估计[J].中国科学:物理学力学天文学,2017,47(09):103-112.
- [17] 梁昊,詹亚锋,尹海亮.X射线脉冲星导航系统选星方法研究[J].电子与信息学报,2015,37(10):2356-2362.
- [18] 尹东山,高玉平,赵书红.综合脉冲星时间尺度[J].天文学报,2016,57(03):326-335.

附录 A Python 源程序

A.1 第 1 问程序

q1.py

```
import numpy as np
import math
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文字体为黑体
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

# 常量
GM = 398600.4418 # 地球引力常量 unit : km3/s2

def ComputePosVelFromElements(e, h, i,  $\theta$ ,  $\Omega$ ,  $\omega$ ):
    """由卫星轨道根数计算卫星位置和速度"""
    a = h**2/(GM*(1-e**2)) # 长半轴 unit : km
    r = a*(1-e**2)/(1+e*math.cos( $\theta$ )) # 卫星至地心的距离 unit : km
    p = a*(1-e**2) # 半通径
    P = np.array([ math.cos( $\omega$ )*math.cos( $\Omega$ ) - math.sin( $\omega$ )*math.cos(i)*math.sin( $\Omega$ ),
                  math.cos( $\omega$ )*math.sin( $\Omega$ ) + math.sin( $\omega$ )*math.cos(i)*math.cos( $\Omega$ ),
                  math.sin( $\omega$ )*math.sin(i)])
    Q = np.array([-math.sin( $\omega$ )*math.cos( $\Omega$ ) - math.cos( $\omega$ )*math.cos(i)*math.sin( $\Omega$ ),
                  -math.sin( $\omega$ )*math.sin( $\Omega$ ) + math.cos( $\omega$ )*math.cos(i)*math.cos( $\Omega$ ),
                  math.cos( $\omega$ )*math.sin(i)])

    r_vector = r*math.cos( $\theta$ )*P + r*math.sin( $\theta$ )*Q # unit: km
    v_vector = -h/p*math.sin( $\theta$ )*P + h/p*(e+math.cos( $\theta$ ))*Q # unit: km/s
    return r_vector[0], r_vector[1], r_vector[2], v_vector[0], v_vector[1], v_vector[2]

def ComputeElementsFromPosVel(r_x, r_y, r_z, v_x, v_y, v_z):
```



```

"""由卫星位置和速度计算卫星轨道根数"""
r_vector = np.array([r_x, r_y, r_z])
v_vector = np.array([v_x, v_y, v_z])
r = np.linalg.norm(r_vector, ord=2)
v = np.linalg.norm(v_vector, ord=2)

h_vector = np.cross(r_vector, v_vector)
h = np.linalg.norm(h_vector, ord=2)
Wx = h_vector[0]/h
Wy = h_vector[1]/h
Wz = h_vector[2]/h

i = math.atan2(math.sqrt(Wx**2+Wy**2), Wz) # 轨道倾角
if i < 0:
    i = i + math.pi*2.0
Ω = math.atan2(Wx, -Wy) # 升交点赤经
if Ω < 0:
    Ω = Ω + math.pi*2.0
p = h*h/GM # 半通径
a = (2/r - v*v/GM)**-1 # 半长轴
n = math.sqrt(GM/a**3) # 平运动角速度
e = math.sqrt(1 - p/a) # 偏心率
E = math.atan2(np.dot(r_vector, v_vector)/(a**2*n), 1-r/a) #
    偏近点角
if E < 0:
    E = E + math.pi*2.0
M = E - e*math.sin(E) # 平近点角
u = math.atan2(r_z, -r_x*Wy+r_y*Wx) # 纬度辐角
if u < 0:
    u = u + math.pi*2.0
Θ = math.atan2(math.sqrt(1-e**2)*math.sin(E), math.cos(E)-
    e) # 真近点角
if Θ < 0:
    Θ = Θ + math.pi*2.0
ω = u - Θ # 近地点辐角
if ω < 0:
    ω = ω + math.pi*2.0
return e, h, Ω, i, ω, Θ, a

```

```

def main():
    """主函数"""
    ## 已知量
    e = 2.06136076e-3
    h = 5.23308462e4
    i = 1.69931232
     $\theta$  = 3.43807372
     $\Omega$  = 5.69987423
     $\omega$  = 4.10858621

    # 正算 - 计算位置和速度
    r_x, r_y, r_z, v_x, v_y, v_z = ComputePosVelFromElements(e
        , h, i,  $\theta$ ,  $\Omega$ ,  $\omega$ )
    print(f"由轨道根数到卫星位置和速度: ")
    print(f"卫星位置(km):x = {r_x}, y = {r_y}, z = {r_z}")
    print(f"卫星速度(km/s):vx = {v_x}, vy = {v_y}, vz = {v_z}"
        )

    # 验证: 反算 - 计算六个轨道根数
    e1, h1,  $\Omega$ 1, i1,  $\omega$ 1,  $\theta$ 1, a1 = ComputeElementsFromPosVel(r_x
        , r_y, r_z, v_x, v_y, v_z)
    print(f"由卫星位置和速度到轨道根数: ")
    print(f"e : {e1}")
    print(f"h : {h1}")
    print(f" $\Omega$  : { $\Omega$ 1}")
    print(f"i : {i1}")
    print(f" $\omega$  : { $\omega$ 1}")
    print(f" $\theta$  : { $\theta$ 1}")

    # 验证: 卫星近日点和远日点之间的距离应该等于2倍卫星轨道的
    长半轴
     $\theta$  = 0.0 # 卫星近日点
    r_x1, r_y1, r_z1, v_x1, v_y1, v_z1 =
        ComputePosVelFromElements(e, h, i,  $\theta$ ,  $\Omega$ ,  $\omega$ )
    r_near = np.sqrt(r_x1**2+r_y1**2+r_z1**2)
     $\theta$  = math.pi # 卫星远日点
    r_x2, r_y2, r_z2, v_x2, v_y2, v_z2 =
        ComputePosVelFromElements(e, h, i,  $\theta$ ,  $\Omega$ ,  $\omega$ )
    r_far = np.sqrt(r_x2**2+r_y2**2+r_z2**2)

```

```

print("验证：卫星近日点和远日点之间的距离应该等于2倍卫星轨道的长半轴")
print(f"r_near*0.5+r_far*0.5-a (m): {(r_near*0.5+r_far*0.5-a1)*1000.0:.5f} ")

## 卫星轨道外推以及位置可视化
radius = 6371
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
u = np.linspace(0, 2 * np.pi, 500)
v = np.linspace(0, np.pi, 500)
x = radius * np.outer(np.cos(u), np.sin(v))
y = radius * np.outer(np.sin(u), np.sin(v))
z = radius * np.outer(np.ones(np.size(u)), np.cos(v))
ax.plot_surface(x, y, z, color='blue')
# 把 $\theta$ 真近点角外推 $[0-2\pi]$ 
 $\theta$  = np.linspace(0, 2*np.pi, 5000)
s_x = []; s_y = []; s_z = []
for j in range(0, len( $\theta$ )):
    r_x_temp, r_y_temp, r_z_temp, v_x_temp, v_y_temp,
    v_z_temp = ComputePosVelFromElements(e, h, i,  $\theta[j]$ ,
     $\Omega$ ,  $\omega$ )
    s_x.append(r_x_temp); s_y.append(r_y_temp); s_z.append(
    r_z_temp)
ax.plot3D(s_x, s_y, s_z, color='red', label="预报轨道") #
    红色折线
ax.scatter3D(r_x, r_y, r_z, color='green', s=50, label="第一
    题的卫星位置")
ax.set_xlabel('X(km)', fontsize=14)
ax.set_ylabel('Y(km)', fontsize=14)
ax.set_zlabel('Z(km)', fontsize=14)
ax.tick_params(axis='x', labelsize=10)
ax.tick_params(axis='y', labelsize=10)
ax.tick_params(axis='z', labelsize=10)
ax.grid(False)
ax.legend()
plt.show()

# Entry
main()

```

A.2 第2问程序

q2.py

```
from jplephem.spk import SPK
import numpy as np
import math

# 常量
c = 299792458 # 真空中的光速 m/s

def TT2TDB(jd_TT):
    """将TT地球时转换为TDB太阳系质心力学时"""
    # 将TT时间转换为TDB时间
    # T = (jd_TT - 2451545.0)/36525 # T为儒略世纪数
    # g = 2*math.pi*(357.528+35999.05*T)/360.0 # g为地球的均角
    # jd_TDB = jd_TT + 0.001658*math.sin(g + 0.0167*math.sin(g))
    g = 6.24+0.017202*(jd_TT - 2451545)
    jd_TDB = jd_TT + 0.001657*math.sin(g)
    return jd_TDB

def SateP_GCRS2SSB(p_sate_GCRS, jd):
    """将卫星位置从地球质心转换到太阳系质心"""
    kernel = SPK.open(rf"F:\shumo\题目\附件\附件3-de200.bsp")
    p_earth_GCRS = kernel[3,399].compute(jd) # 地球质心相对于地月质心的位置 km
    p_earthmoon_SSB = kernel[0,3].compute(jd) # 地月质心相对于太阳系质心SSB的位置 km

    p_earth_SSB = p_earthmoon_SSB + p_earth_GCRS # 地球质心相对于太阳系质心SSB的位置 km
    p_sate_SSB = p_earth_SSB + p_sate_GCRS # 卫星相对于太阳系质心的位置 km

    return p_sate_SSB

def Crab_unit_vector(jd):
    """计算jd时刻，Crab在太阳系质心下的单位向量"""
    ref_jd = 57715.000000295 + 2400000.5
```

```

ref_ra = 83.63307631324167      # ref_jd时刻的赤经 deg
ref_dec = 22.014493269173464    # ref_jd时刻的赤纬 deg
dt_ra = -14.7 * 10**-3 / 3600.0 / 365.2425 # 赤经自行 deg
      /day 默认一年365.2425天
dt_dec= 2.0 * 10**-3 / 3600.0 / 365.2425    # 赤纬自行
      deg/day

ra = ref_ra + (jd - ref_jd) * dt_ra
dec = ref_dec + (jd - ref_jd) * dt_dec
# ra = ref_ra + (ref_jd - ref_jd) * dt_ra
# dec = ref_dec + (ref_jd - ref_jd) * dt_dec
ra = ra/180.0*math.pi    # 弧度
dec = dec/180.0*math.pi

unit_vector = np.array([math.cos(dec)*math.cos(ra),
                        math.cos(dec)*math.sin(ra),
                        math.sin(dec)])

return unit_vector

def main():
    """主函数"""
    # 卫星相对于地球质心下的位置 km
    p_sate_GCRS = np.array([1274.913415085168,
                           -1848.8519649923514, 6507.26265527945])

    # TT时间系统转换为TDB时间系统
    mjd_TT = 57062.0
    jd_TT = mjd_TT + 2400000.5
    jd_TDB = TT2TDB(jd_TT)

    # jd_TDB时刻，将卫星位置从地球质心 转换到 太阳系质心 （中
    # 间考虑了地球质心和地月质心的差异）
    p_sate_SSB = SateP_GCRS2SSB(p_sate_GCRS, jd_TDB) # 卫星在
    # SSB中的位置 km

    # 计算jd_TT时刻，Crab在太阳系质心下的单位向量
    unit_vector = Crab_unit_vector(jd_TT)

    # 计算脉冲星光子到达卫星与太阳系质心的传播路径时间差

```

```

ds = -np.dot(p_sate_SSB, unit_vector)*10**3      # SSB坐标系
        下, 卫星在脉冲星光子方向上的投影距离 m
dt = ds / c                                     # 光子到达卫
        星与太阳系质心的传播路径时间差 s
print(f"传播路径时间差 : {dt} s")

# Entry
main()

```

A.3 第3问程序

q3.py

```

from jplephem.spk import SPK
import numpy as np
import math

# 常量
c = 299792458      # 真空中的光速 m/s

def TT2TDB(jd_TT):
    """将TT时间转换为TDB时间"""
    g = 6.24+0.017202*(jd_TT - 2451545)
    jd_TDB = jd_TT + 0.001657*math.sin(g)
    return jd_TDB

def SatePV_GCRS2SSB(p_sate_GCRS, v_sate_GCRS, jd):
    """将卫星位置和速度从地球质心转换到太阳系质心(考虑地球质心
        和地月质心的差异, 以及地球公转速度和绕地月质心速度)"""
    kernel = SPK.open(rf"F:\shumo\题目\附件\附件3-de200.bsp")
    # 地球质心相对于地月质心的位置 km 和速度 km/s
    p_earth_GCRS, v_earth_GCRS = kernel[3, 399].
        compute_and_differentiate(jd)
    v_earth_GCRS = v_earth_GCRS / 86400.0
    # 地月质心相对于太阳系质心SSB的位置 km 和速度 km/s
    p_earthmoon_SSB, v_earthmoon_SSB = kernel[0, 3].
        compute_and_differentiate(jd)
    v_earthmoon_SSB = v_earthmoon_SSB / 86400.0

    # 地球质心相对于太阳系质心SSB的位置 km 和速度 km/s
    p_earth_SSB = p_earthmoon_SSB + p_earth_GCRS # 地球质心相

```

```

    对于太阳系质心SSB的位置 km
    v_earth_SSB = v_earthmoon_SSB + v_earth_GCRS
    # 卫星相对于太阳系质心的位置 km 和 速度 km/s
    p_sate_SSB = p_earth_SSB + p_sate_GCRS      # 卫星相对于太阳
    系质心的位置 km
    v_sate_SSB = v_earth_SSB + v_sate_GCRS
    return p_sate_SSB, v_sate_SSB

def Crab_unit_vector(jd):
    # 计算jd时刻, Crab在太阳系质心下的单位向量
    ref_jd = 57715.000000295 + 2400000.5      # 参考时刻
    ref_ra = 83.63307631324167      # ref_jd时刻的赤经 deg
    ref_dec = 22.014493269173464      # ref_jd时刻的赤纬 deg
    dt_ra = -14.7 * 10**-3 / 3600.0 / 365.2425      # 赤经自行 deg
    /day 默认一年365.2425天
    dt_dec = 2.0 * 10**-3 / 3600.0 / 365.2425      # 赤纬自行
    deg/day

    ra = ref_ra + (jd - ref_jd) * dt_ra
    dec = ref_dec + (jd - ref_jd) * dt_dec
    ra = ra/180.0*math.pi      # 弧度
    dec = dec/180.0*math.pi

    unit_vector = np.array([math.cos(dec)*math.cos(ra),
                            math.cos(dec)*math.sin(ra),
                            math.sin(dec)])

    return unit_vector

def calculate_geometric_delay(p_sate_SSB, crab_unit_vector):
    # 考虑脉冲星自行下的几何时延
    dt_geometric = -np.dot(p_sate_SSB, crab_unit_vector)*10**3
    / c      # unit : s
    return dt_geometric      # 改正量应该为负

def calculate_shapiro_delay(p_sate_SSB, crab_unit_vector, jd):
    # Shapiro时延, 由于太阳是太阳系内主要的引力源, 因此只考虑
    太阳引力造成的延迟
    GM_sun = 1.327e20      # 太阳引力常数 m3*s-2

```

```

kernel = SPK.open(rf"F:\shumo\题目\附件\附件3-de200.bsp")
p_sun_SSB = kernel[0,10].compute(jd)      # 太阳在太阳系质
      心SSB中的位置 km
p_sun2sate_SSB = p_sate_SSB - p_sun_SSB    # SSB中, 卫星相对
      太阳的位置 km

d_pro_crab = np.dot(p_sun2sate_SSB, crab_unit_vector)
      *10**3 # 在carb光子视线上的投影 m
d_sate2sun = np.linalg.norm(p_sun2sate_SSB, ord=2)*10**3
      # 卫星相对于太阳的距离 m
dt_shapiro = -2.0*GM_sun/c**3*np.log(d_pro_crab/d_sate2sun
      +1) # unit : s

return dt_shapiro # 改正量应该为负

def calculate_redshift_delay(p_sate_SSB, jd):
    # 引力红移时延, 由于太阳是太阳系内主要的引力源, 因此只考虑
      太阳引力造成的延迟
    # 假设以太阳作为主要引力源, 并且r是卫星到太阳系质心的距离
    GM_sun = 1.327e20 # 太阳引力常数 m3*s-2

    ## 这个是按照卫星到太阳的距离
    # kernel = SPK.open(rf"F:\shumo\题目\附件\附件3-de200.bsp
      ")
    # p_sun_SSB = kernel[0,10].compute(jd) # 太阳在SSB中的位
      置 km
    # p_sate2sun_SSB = p_sate_SSB - p_sun_SSB # 卫星相对太
      阳的位置 km
    # d_sate = np.linalg.norm(p_sate2sun_SSB, ord=2)*10**3 #
      卫星相对于太阳的距离 m

    d_sate = np.linalg.norm(p_sate_SSB, ord=2)*10**3 # 卫星
      到太阳系质心的距离 m
    dt_redshift = (-GM_sun/d_sate)/c**2 # unit
      : s

    return dt_redshift # 改正量应该为负

def calculate_specialrelativity_delay(p_sate, v_sate):

```



```

# 狭义相对论的动钟变慢效应
dt_specialrelativity = 2.0 * np.dot(p_sate*10**3, v_sate
    *10**3) / c**2 # unit : s
return dt_specialrelativity

def main():
    # # 卫星相对于地球质心下的位置 km 和 速度 km/s
    p_sate_GCRS = np.array([1274.913415085168,
        -1848.8519649923514, 6507.26265527945])
    v_sate_GCRS = np.array([-6.210795173831417,
        3.7461273059407443, 2.2763308795948665])

    # TT时间系统转换为TDB时间系统
    mjd_TT = 58119.1651507519
    jd_TT = mjd_TT + 2400000.5
    jd_TDB = TT2TDB(jd_TT)

    # jd_TDB时刻, 将卫星位置从地球质心 转换到 太阳系质心 (中
        间考虑了地球质心和地月质心的差异)
    p_sate_SSB, v_sate_SSB = SatePV_GCRS2SSB(p_sate_GCRS,
        v_sate_GCRS, jd_TDB)

    # 计算jd_TT时刻, Crab在太阳系质心下的单位向量
    unit_vector = Crab_unit_vector(jd_TT)

    # 计算各项时延
    dt_geometric = calculate_geometric_delay(p_sate_SSB,
        unit_vector)
    dt_shapiro = calculate_shapiro_delay(p_sate_SSB,
        unit_vector, jd_TDB)
    dt_redshift = calculate_redshift_delay(p_sate_SSB, jd_TDB)
    # dt_specialrelativity = calculate_specialrelativity_delay
        (p_sate_SSB, v_sate_SSB)
    dt_specialrelativity = calculate_specialrelativity_delay(
        p_sate_GCRS, v_sate_GCRS)
    dt = dt_geometric+dt_shapiro+dt_redshift+
        dt_specialrelativity
    print(f"几何延迟: {dt_geometric} s")
    print(f"shapiro效应延迟: {dt_shapiro} s")

```

```

print(f"红移效应延迟: {dt_redshift} s")
print(f"狭义相对论延迟: {dt_specialrelativity} s")
print(f"总延迟: {dt} s")

# Entry
main()

```

A.4 第4问1、2小问程序

q4₁₂.py

```

import numpy as np
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt
from scipy.interpolate import CubicSpline
import random
from scipy.integrate import simpson

plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文字体为黑体
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

def readfl(file):
    """读取标准轮廓曲线信息"""
    xy = np.loadtxt(file)
    x = xy[:, 0]
    y = xy[:, 1]
    return x, y

def fun(x, a, b, c):
    """定义前三个分段指数函数"""
    return a*np.exp(b*x)+c

def fun4(x, a, b, c, d):
    """定义第四个分段指数函数"""
    # return a*np.exp(b*(x-0.92))+c
    return a*np.exp(b*(x-d))+c

def fit(x, y, x1, x2):
    """拟合前三个分段指数函数"""
    flag = (x1 < x) & (x <= x2)
    x_fit = x[flag]

```

```

y_fit = y[flag]
popt, pcov = curve_fit(fun, x_fit, y_fit)
# 生成拟合的结果
x_fit = np.arange(x1,x2,0.001)
y_fit = fun(x_fit, pop[0], pop[1], pop[2])
return pop,x_fit,y_fit

def fit4(x,y,x1,x2):
    """拟合第四个分段指数函数"""
    flag = (x1 < x) & (x <= x2)
    x_fit = x[flag]
    y_fit = y[flag]
    pop, pcov = curve_fit(fun4, x_fit, y_fit)
    # 生成拟合结果
    x_fit = np.arange(x1,x2,0.001)
    y_fit = fun4(x_fit, pop[0], pop[1], pop[2], pop[3])
    return pop,x_fit,y_fit

def get_φ(t,dt, t0):
    """得到t时刻的相位"""
    t = t + (-dt)          # dt为时延, 单位: s
    φ_t0 = 0               # φ_t0为初始相位
    v = 29.647854750036593 # 参考时刻的自转频率 (s-1)
    dv1 = -368970.96e-15   # 参考时刻的自转频率一阶导数(s-2)
    v = v + dv1*t0         # 从参考时刻归算到当前时刻的自转频率 (s-1)
    φ = φ_t0 + v*(t - t0) + 0.5*dv1*(t - t0)**2
    φ = φ % 1              # 将得到的相位转化为[0,1]之间
    return φ

def get_h_LinearInter(φ,φ_x,h_y):
    """根据标准轮廓进行线性插值得到φ相位下对应的h"""
    diff = np.abs(φ_x - φ)
    closest_index1 = np.argmin(diff)
    closest_index2 = 0
    if φ - φ_x[closest_index1] >= 0:
        closest_index2 = closest_index1 + 1
    else:
        closest_index2 = closest_index1 - 1

```

```

        closest_index1, closest_index2 = closest_index2,
        closest_index1

## 线性插值
closest_index1 = int(closest_index1)
closest_index2 = int(closest_index2)
if closest_index1 == -1:
    h_φ = h_y[0]
    return h_φ
if closest_index2 == len(h_y):
    h_φ = h_y[-1]
    return h_φ
h1 = h_y[closest_index1]
h2 = h_y[closest_index2]
φ1 = φ_x[closest_index1]
φ2 = φ_x[closest_index2]
h_φ = (h2 - h1) / (φ1 - φ2) * (φ - φ1) + h1
return h_φ

def get_h_SegfunctionFit(φ, popt1, popt2, popt3, popt4, x1, x2, x3, x4
, x5):
    """根据拟合的分段指数函数得到φ相位下对应的h"""
    if x1 <= φ <= x2:
        h = fun(φ, popt1[0], popt1[1], popt1[2])
        return h
    elif x2 < φ <= x3:
        h = fun(φ, popt2[0], popt2[1], popt2[2])
        return h
    elif x3 < φ <= x4:
        h = fun(φ, popt3[0], popt3[1], popt3[2])
        return h
    elif x4 < φ <= x5:
        h = fun4(φ, popt4[0], popt4[1], popt4[2], popt4[3])
        return h
    else:
        print(f"error :x1 < φ <= x2, φ = {φ}")
        exit(1)

def get_h_CubicSplineInter(φ, cs):

```

```

    """根据三次样条插值得到 $\varphi$ 相位下对应的h"""
    h = cs( $\varphi$ )
    return h

def get_ $\lambda$ (h, s):
    """得到在探测器有效面积s下的流量"""
     $\lambda_b$  = 1.54*s # ph/s
     $\lambda_s$  = 10* $\lambda_b$ 
     $\lambda$  =  $\lambda_b$  +  $\lambda_s$  * h
    return  $\lambda$ 

def get_Pearson(s, w):
    """计算模拟轮廓s与标准轮廓w的Pearson系数"""
    len_w = len(w)
    correlation_matrix = np.corrcoef(s[0:len_w], w)
    pearson = correlation_matrix[0,1]
    return pearson

def main():
    """主函数"""
    # 读取标准轮廓曲线数据
    file = rf"F:\shumo\题目\附件\附件2: 标准轮廓曲线.txt"
     $\varphi_x$ , h_y = readfl(file)
    inter_method = '三次样条'

    ## 分段指数函数拟合
    x1=0;x2=0.2;x3=0.4;x4=0.82;x5=1.0
    popt1,x_fit1,y_fit1 = fit( $\varphi_x$ ,h_y,x1,x2)
    popt2,x_fit2,y_fit2 = fit( $\varphi_x$ ,h_y,x2,x3)
    popt3,x_fit3,y_fit3 = fit( $\varphi_x$ ,h_y,x3,x4)
    popt4,x_fit4,y_fit4 = fit4( $\varphi_x$ ,h_y,x4,x5)
    ## 拟合结果绘图
    fig, ax1 = plt.subplots(1, 1, sharex=True, figsize=(10, 5)
        )
    ax1.plot( $\varphi_x$ ,h_y,c='b',label='标准脉冲轮廓')
    ax1.plot(x_fit1,y_fit1,label='分段指数函数1')
    ax1.plot(x_fit2,y_fit2,label='分段指数函数2')
    ax1.plot(x_fit3,y_fit3,label='分段指数函数3')
    ax1.plot(x_fit4,y_fit4,label='分段指数函数4')

```

```

ax1.set_xlabel('相位', fontsize=14)
ax1.set_ylabel('脉冲轮廓h', fontsize=14)
ax1.tick_params(axis='x', labelsz=14)
ax1.tick_params(axis='y', labelsz=14)
ax1.legend(loc='upper center')
plt.savefig('4_1-分段指数函数拟合.pdf', pad_inches=None,
           bbox_inches='tight', dpi=300)
plt.show()

## 三次样条插值
cs = CubicSpline( $\phi_x$ , h_y)
 $\phi_{cs}$  = np.linspace(0, 1, 1024)
h_cs = cs( $\phi_{cs}$ )
## 拟合结果绘图
fig, ax1 = plt.subplots(1, 1, sharex=True, figsize=(10, 5)
)
ax1.plot( $\phi_x$ , h_y, c='b', label='标准脉冲轮廓')
ax1.plot( $\phi_{cs}$ , h_cs, c='r', label='三次样条插值拟合')
ax1.set_xlabel('相位', fontsize=14)
ax1.set_ylabel('脉冲轮廓h', fontsize=14)
ax1.tick_params(axis='x', labelsz=14)
ax1.tick_params(axis='y', labelsz=14)
ax1.legend(loc='upper center')
plt.savefig('4_1-三次样条插值拟合.pdf', pad_inches=None,
           bbox_inches='tight', dpi=300)
plt.show()

## 问题4 - (1) 生成模拟时间序列
t_seris = []      # 记录仿真时间为10s的光子时间序列，以t0为
                  # 起始时刻
s = 250           # 探测器的有效面积(cm2)
t0 = 0            # 起始时刻或与参考时刻之差(s)
t_obs = 10.0      # 持续时间(s)
t_e = t0 + t_obs  # 结束时刻(s)
dt = 0            # 时延(s)
t_k = t0
random.seed(10)
while (t_k <= t_e):
    y = random.random()    # 区间(0,1)的一个随机变量y

```

```

     $\varphi_t$  = get_ $\varphi$ (t_k,dt, t0)
    # h_t = get_h_SegfunctionFit( $\varphi_t$ ,popt1,popt2,popt3,
        popt4,x1,x2,x3,x4,x5) # 分段指数函数拟合
    # h_t = get_h_LinearInter( $\varphi_t$ , $\varphi_x$ ,h_y) # 线性内
        插
    h_t = get_h_CubicSplineInter( $\varphi_t$ ,cs) # 三次样条函数
        插值
     $\lambda_t$  = get_ $\lambda$ (h_t, s)
    t_k1 = t_k + (-np.log(y)/ $\lambda_t$ ) # t_(k+1)与t_k 时刻之
        间的递推
    # print("t_k:",t_k1," $\varphi$ :", $\varphi_t$ ,"y:",y," $\lambda_t$ :", $\lambda_t$ ,"h_t",
        h_t)
    t_seris.append(t_k1)
    t_k = t_k1

## 光子到达时间分布直方图
fig, ax1 = plt.subplots(1, 1, sharex=True, figsize=(10, 5)
    )
ax1.hist(t_seris, bins=200, edgecolor='black')
ax1.set_title('光子到达时间分布直方图',fontsize=14)
ax1.set_xlabel('光子到达时间(s)',fontsize=14)
ax1.set_ylabel('光子到达的个数',fontsize=14)
ax1.set_xlim(-0.1,10.1)
ax1.tick_params(axis='x', labels=14)
ax1.tick_params(axis='y', labels=14)
plt.savefig(f'4_1-{inter_method}-光子到达时间分布直方图.
    pdf', pad_inches=None, bbox_inches='tight',dpi=300)
plt.show()

# 问题4 - (2)脉冲星光子序列折叠出脉冲轮廓
v = 29.647854750036593 # 自转频率(s-1)
T = 1.0 / v # 周期(s)
t_seris = np.mod(t_seris, T) # 归算到一个周期内
bin = 256 # 一个周期内的子相位间隔数
interval = T / bin # 子相位时间间隔(s)
x = np.arange(0, T+interval*0.1,interval)
num = np.zeros(len(x))
for i in range(0,len(t_seris)):
    diff = np.abs(x - t_seris[i])

```

```

        closest_index = np.argmin(diff)
        num[closest_index] = num[closest_index] + 1
x = x/T # 将相位变为[0,1]
num[0] = num[0] + num[-1]
num = num[0:len(num)-1]
x = x[0:len(x)-1]

## 原始的模拟光子轮廓
fig, ax2 = plt.subplots(1, 1, sharex=True, figsize=(10, 5)
    )
ax2.plot(x, num, c='red', label='仿真脉冲轮廓')
ax2.set_xticks(np.arange(0, 1.01, 0.2))
ax2.set_title('仿真时间10s时光子数据折叠的脉冲轮廓',
    fontsize=14)
ax2.set_xlabel('相位', fontsize=14)
ax2.set_ylabel('光子个数(个)', fontsize=14)
ax2.tick_params(axis='x', labelsz=14)
ax2.tick_params(axis='y', labelsz=14)
ax2.legend(loc='upper center')
plt.savefig(f'4_1-{inter_method}-仿真时间10s光子数据折叠的
    脉冲轮廓.pdf', pad_inches=None, bbox_inches='tight', dpi
    =300)
plt.show()

## 模拟的轮廓与标准的轮廓进行匹配, 并计算pearson系数
num = num /.simps(num, x) *simps(h_y,  $\phi_x$ )
s = []
for i in range(0, len(num)):
    s.append(num[i])
for i in range(0, len(num)):
    s.append(num[i])
s = np.array(s)
pearson = []
for i in range(0, len(num)):
    pearson.append(get_Pearson(s[i:], h_y))
pearson_max = np.max(pearson)
pearson_max_shift = np.argmax(pearson)
print(f"{inter_method} 模拟轮廓与标准轮廓的pearson系数: {
    pearson_max}")

```



```

s = s[pearson_max_shift:len(h_y)+pearson_max_shift]

## 仿真脉冲轮廓和标准脉冲轮廓可视化
fig, ax3 = plt.subplots(1, 1, sharex=True, figsize=(10, 5)
)
ax3.plot( $\phi_x$ , s, c='red', label='仿真脉冲轮廓')
ax3.plot( $\phi_x$ , h_y, c='blue', label='标准脉冲轮廓')
ax3.set_xticks(np.arange(0, 1.01, 0.2))
ax3.set_xlabel('相位', fontsize=14)
ax3.set_ylabel('脉冲轮廓', fontsize=14)
ax3.tick_params(axis='x', labels=14)
ax3.tick_params(axis='y', labels=14)
ax3.legend(loc='upper center')
plt.savefig(f'4_2-{{inter_method}}-仿真脉冲轮廓和标准脉冲轮廓.pdf', pad_inches=None, bbox_inches='tight', dpi=300)
plt.show()

main()

```

A.5 第4问1、2小问检验程序

q4₁₂test.py

```

import numpy as np
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt
from scipy.interpolate import CubicSpline
import random
import scipy.stats as stats
from scipy.integrate import simps

plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文字体为黑体
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

def readfl(file):
    """读取标准轮廓曲线信息"""
    xy = np.loadtxt(file)
    x = xy[:, 0]
    y = xy[:, 1]
    return x, y

```

```

def get_φ(t, dt, t0):
    """得到t时刻的相位"""
    t = t + (-dt)          # 单位: s
    φ_t0 = 0               # φ_t0为初始相位
    v = 29.647854750036593 # 参考时刻的自转频率 (s-1)
    dv1 = -368970.96e-15   # 参考时刻的自转频率一阶导数 (s-2)
    v = v + dv1*t0         # 从参考时刻归算到当前时刻的自转频率 (s-1)
    φ = φ_t0 + v*(t - t0) + 0.5*dv1*(t - t0)**2
    φ = φ % 1.0           # 将得到的相位转化为[0,1]之间
    return φ

def get_h_CubicSplineInter(φ, cs):
    """根据三次样条插值得到φ相位下对应的h"""
    h = cs(φ)
    return h

def get_λ(h, s):
    """得到在探测器有效面积s下的流量"""
    λb = 1.54*s   # ph/s
    λs = 10*λb
    λ = λb + λs * h
    return λ

def fun_p(λ, k):
    """利用泊松分布计算概率"""
    sum1 = 0.0
    for i in range(1, k+1):
        sum1 = sum1 + np.log(i)
    ln_y = k*np.log(λ) - λ - sum1

    return np.exp(ln_y)

def main():
    """主函数"""
    # 读取标准轮廓曲线数据
    file = rf"F:\shumo\题目\附件\附件2: 标准轮廓曲线.txt"
    φ_x, h_y = readfl(file)

```

```

## 三次样条插值
cs = CubicSpline( $\varphi_x$ , h_y)

## 对模拟数据的泊松过程进行统计检验
n = 30    # 检测的模拟周期(个) 30个周期时间跨度为0-1.012s
v = 29.647854750036593    # 自转频率(s-1)
T = 1.0 / v    # 周期(s)
s = 250    # 有效面积(cm2)
 $\lambda_b$  = 1.54*s    # 背景光子流量密度(ph/s)
 $\lambda_s$  = 10* $\lambda_b$     # 脉冲星光子流量密度(ph/s)
E = n*( $\lambda_b$ *T +  $\lambda_s$ /v*simps(h_y,  $\varphi_x$ ))    # 在检验n个周期情况下, 计算皮尔逊统计量(30个周期时,E为4283)
i_max = int(E)*2    # 样本最大值

## 生成i_max个样本
sample_data = []
for i in range(0,i_max):
    t_seris = []    # 记录仿真时间为10s的光子时间序列, 以
                    # t0为起始时刻
    s = 250    # 探测器的有效面积(cm2)
    t0 = 0    # 起始时刻(s)
    t_obs = T*30.0    # 持续时间(s)
    t_e = t0 + t_obs    # 结束时刻(s)
    dt = 0    # 时延(s)
    t_k = t0
    while (t_k <= t_e):
        y = random.random()    # 区间(0,1)的一个均匀随机变量y
         $\varphi_t$  = get_ $\varphi$ (t_k, dt, t0)
        # h_t = get_h_SegfunctionFit( $\varphi_t$ , popt1, popt2, popt3
        # , popt4, x1, x2, x3, x4, x5)    # 分段指数函数拟合
        # h_t = get_h_LinearInter( $\varphi_t$ ,  $\varphi_x$ , h_y)    # 线性函数拟合
        h_t = get_h_CubicSplineInter( $\varphi_t$ , cs)    # 三次样条插值
         $\lambda_t$  = get_ $\lambda$ (h_t, s)
        t_k1 = t_k + (-np.log(y)/ $\lambda_t$ )    # t_(k+1)与t_k 时刻之间的递推

```

```

        # print("t_k:", t_k1, "φ:", φ_t, "y:", y, "λ_t:", λ_t, "
        h_t", h_t)
        t_seris.append(t_k1)
        t_k = t_k1
    print(f"\ri+1={i+1} {len(t_seris)}", end='')
    sample_data.append(len(t_seris))

sample_data = np.array(sample_data)
# 统计样本出现的结果以及对应出现的次数
sample_values, sample_counts = np.unique(sample_data,
    return_counts=True)

## 模拟样本光子数目分布图
fig, ax1 = plt.subplots(1, 1, sharex=True, figsize=(10, 5)
    )
ax1.scatter(sample_values, sample_counts)
ax1.set_xlabel('光子到达数目(个)', fontsize=14)
ax1.set_ylabel('对应的样本数量', fontsize=14)
ax1.set_yticks(np.arange(0, 70.1, 10))
ax1.set_ylim(-1, 70)
ax1.tick_params(axis='x', labelsize=14)
ax1.tick_params(axis='y', labelsize=14)
plt.savefig('4_1-三次样条-假设检验_模拟样本光子数目分布图.
    pdf', pad_inches=None, bbox_inches='tight', dpi=300)
plt.show()

## 计算每种情况出现的概率
outfile2 = "4_1 概率.txt"
a = []
p = []

for i in range(0, i_max-1):
    p_temp = fun_p(E, i)
    a.append(i)
    p.append(p_temp)
    print(f"\r i={i} {p_temp}", end='')
a = np.array(a)
p = np.array(p)

```

```

ap_data = np.loadtxt(outfile2)
a = ap_data[:,0]
p = ap_data[:,1]
a = np.append(a, a[-1]+1)
p = np.append(p, 1-np.sum(p))
# 计算统计检验量
chi2_stat = 0.0
n = np.sum(sample_counts)
for i in range(0, len(a)):
    indices = np.argwhere(sample_values == a[i])
    if len(indices) > 0:
        indices = indices[0][0]
        chi2_stat = chi2_stat + (sample_counts[indices]-n*
            p[i])**2/n*p[i]
    else:
        chi2_stat = chi2_stat + n*p[i]

## 计算卡方分布的临界值  $X_{(1-\alpha)}(k)$ 
alpha = 0.05 # 显著性水平alpha
k = len(sample_values)-1 # 自由度k(根据样本出现的情况设定)
chi2_critical = stats.chi2.ppf(1 - alpha, df=k)
print(f"自由度:", k)
print(f"统计检验量: {chi2_stat}")
print(f"卡方分布的临界值: ", chi2_critical)
# 结论
if chi2_stat < chi2_critical:
    print("结论: 本文产生的光子到达时刻服从泊松分布")
else:
    print("结论: 本文产生的光子到达时刻不服从泊松分布")

main()

```

A.6 第4问3小问程序

q4₃.py

```

import numpy as np
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt
from scipy.interpolate import CubicSpline

```

```

import random
from scipy.integrate import simp
plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文字体为
黑体
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

def readfl(file):
    """读取标准轮廓曲线信息"""
    xy = np.loadtxt(file)
    x = xy[:, 0]
    y = xy[:, 1]
    return x, y

def get_φ(t, dt, t0):
    """得到t时刻的相位"""
    t = t + (-dt) # 单位: s
    φ_t0 = 0 # φ_t0为初始相位
    v = 29.647854750036593 # 参考时刻的自转频率 (s-1)
    [57715.0000000295]
    dv1 = -368970.96e-15 # 参考时刻的自转频率一阶导数(s-2)
    v = v + dv1*t0 # 从参考时刻归算到当前时刻的自转频
    率(s-1)
    φ = φ_t0 + v*(t - t0) + 0.5*dv1*(t - t0)**2
    φ = φ % 1 # 将得到的相位转化为[0,1]之间
    return φ

def get_h_CubicSplineInter(φ, cs):
    """根据三次样条插值得到φ相位下对应的h"""
    h = cs(φ)
    return h

def get_λ(h, s):
    """得到在探测器有效面积s下的流量"""
    λb = 1.54*s # ph/s
    λs = 10*λb
    λ = λb + λs * h
    return λ

def get_Λ(K, t_sample, t0, dt, s, cs):

```

```

"""计算离散积分速率函数 $\Lambda_{t_k}$ """
 $\Lambda_{t_k}$  = []
 $\Lambda_{t_0}$  = 0.0
 $\Lambda_{t_k}$ .append( $\Lambda_{t_0}$ )    # unit : pb
t_k = []    # unit : s
t_k.append(t0)
for i in range(1, K+1):
    t_k.append(t_k[-1]+t_sample)
for i in range(1, K+1):
     $\varphi$  = get_ $\varphi$ (t_k[i], dt, t0)
    h = get_h_CubicSplineInter( $\varphi$ , cs)
     $\Lambda_{t_k}$ .append( $\Lambda_{t_k}$ [-1]+get_ $\lambda$ (h, s)*t_sample)
return  $\Lambda_{t_k}$ , t_k

def scaling( $\Lambda_{t_k}$ ,  $\tau_i$ , t_k_list, t_sample):
    """使用尺度变换得到对应的非齐次泊松过程到达时间"""
    diff = np.abs( $\Lambda_{t_k}$  -  $\tau_i$ )
    closest_index1 = np.argmin(diff)
    closest_index2 = 0
    if  $\tau_i$  -  $\Lambda_{t_k}$ [closest_index1] >= 0:
        closest_index2 = closest_index1 + 1
    else:
        closest_index2 = closest_index1 - 1
        closest_index1, closest_index2 = closest_index2,
        closest_index1

    ## 线性插值
    t_i = 0.0
    closest_index1 = int(closest_index1)
    closest_index2 = int(closest_index2)
    if closest_index1 == -1:
        t_i = t_k_list[0]
        return t_i
    if closest_index2 == len( $\Lambda_{t_k}$ ):
        t_i = t_k_list[-1]
        return t_i
     $\Lambda_1$  =  $\Lambda_{t_k}$ [closest_index1]
     $\Lambda_2$  =  $\Lambda_{t_k}$ [closest_index2]
    t_i = t_k_list[closest_index1] + ( $\tau_i$ - $\Lambda_1$ )/( $\Lambda_2$ - $\Lambda_1$ )*t_sample

```

```

return t_i

def get_Pearson(s, w):
    """ 计算模拟轮廓s与标准轮廓w的Pearson系数 """
    len_w = len(w)
    correlation_matrix = np.corrcoef(s[0:len_w], w)
    pearson = correlation_matrix[0,1]
    return pearson

def main():
    """ 主函数 """
    # 读取标准轮廓曲线数据
    file = rf"F:\shumo\题目\附件\附件2: 标准轮廓曲线.txt"
    phi_x, h_y = readfl(file)
    cs = CubicSpline(phi_x, h_y) # 三次样条插值

    # 问题4 - (3) 生成模拟时间序列
    v = 29.647854750036593 # 自转频率 (s-1)
    T = 1.0 / v # 周期 (s)
    bin = 256 # 一个周期内的子相位间隔数
    t_seris = [] # 记录仿真时间为10s的光子时间序列, 以t0为
    # 起始时刻
    s = 250 # 探测器的有效面积 (cm2)
    t0 = 0 # 起始时刻或与参考时刻之差 (s)
    t_obs = 10.0 # 持续时间 (s)
    t_e = t0 + t_obs # 结束时刻 (s)
    dt = 0 # 时延 (s)
    t_sample = T / bin # 采样间隔 (s)
    K = int((t_e - t0)/t_sample)

    ## 获取离散积分速率函数
    Lambda_t_k, t_k_list = get_Lambda(K, t_sample, t0, dt, s, cs)
    Lambda_K = Lambda_t_k[-1]
    np.random.seed(42)
    # 产生一个服从参数为Lambda_K的泊松随机数I
    I = np.random.poisson(Lambda_K)
    print(f"泊松随机数I: {I}")
    e_i_list = np.random.uniform(0, 1, I) # 生成I个服从 U(0,

```



```

1) 的随机变量
 $\tau_i$ _list =  $\Lambda_K$ *e_i_list

t_i = []
for i in range(0, I):
    print(f"\rk = {i}", end='')
    t_temp = scaling( $\Lambda_{t_k}$ ,  $\tau_i$ _list[i], t_k_list, t_sample)
    t_i.append(t_temp)
t_seris = np.sort(t_i)  # 从小到大的顺序进行排列, 得到光子
    到达时间序列

# 光子到达时间分布直方图
fig, ax1 = plt.subplots(1, 1, sharex=True, figsize=(10, 5)
    )
ax1.hist(t_seris, bins=200, edgecolor='black')
ax1.set_title('光子到达时间分布直方图', fontsize=14)
ax1.set_xlabel('光子到达时间(s)', fontsize=14)
ax1.set_xlim(-0.1, 10.1)
ax1.set_ylabel('光子到达的个数', fontsize=14)
ax1.tick_params(axis='x', labelsize=14)
ax1.tick_params(axis='y', labelsize=14)
plt.savefig('4_3光子到达时间分布直方图.pdf', pad_inches=
    None, bbox_inches='tight', dpi=300)
plt.show()

## 问题4-(3)模拟的脉冲星光子序列折叠出脉冲轮廓
v = 29.647854750036593  # 自转频率(s-1)
T = 1.0 / v  # 周期(s)
t_seris = np.mod(t_seris, T)  # 归算到一个周期内
bin = 256  # 一个周期内的子相位间隔数
inteval = T / bin  # 子相位时间间隔
x = np.arange(0, T+inteval*0.1, interval)
num = np.zeros(len(x))
for i in range(0, len(t_seris)):
    diff = np.abs(x - t_seris[i])
    closest_index = np.argmin(diff)
    num[closest_index] = num[closest_index] + 1
x = x/T  # 将转化为[0, 1]
num[0] = num[0] + num[-1]

```

```

num = num[0:len(num)-1]
x = x[0:len(x)-1]

## 原始的模拟光子轮廓可视化
fig, ax2 = plt.subplots(1, 1, sharex=True, figsize=(10, 5)
)
ax2.plot(x, num, c='red', label='仿真脉冲轮廓')
ax2.set_xticks(np.arange(0, 1.01, 0.2))
ax2.set_title('仿真时间10s时光子数据折叠的脉冲轮廓',
              fontsize=14)
ax2.set_xlabel('相位', fontsize=14)
ax2.set_ylabel('光子个数(个)', fontsize=14)
ax2.tick_params(axis='x', labels=14)
ax2.tick_params(axis='y', labels=14)
ax2.legend(loc='upper center')
plt.savefig('4_3仿真时间10s光子数据折叠的脉冲轮廓.pdf',
            pad_inches=None, bbox_inches='tight', dpi=300)
plt.show()

## 模拟的轮廓与标准的轮廓进行匹配，并计算pearson系数
num = num /.simps(num, x) *simps(h_y,  $\phi_x$ )
s = []
for i in range(0, len(num)):
    s.append(num[i])
for i in range(0, len(num)):
    s.append(num[i])
s = np.array(s)
pearson = []
for i in range(0, len(num)):
    pearson.append(get_Pearson(s[i:], h_y))
pearson_max = np.max(pearson)
pearson_max_shift = np.argmax(pearson)
print(f"模拟轮廓与标准轮廓的pearson系数: {pearson_max}")
s = s[pearson_max_shift:len(h_y)+pearson_max_shift]

## 仿真脉冲轮廓和标准脉冲轮廓可视化
fig, ax3 = plt.subplots(1, 1, sharex=True, figsize=(10, 5)
)
ax3.plot( $\phi_x$ , s, c='red', label='仿真脉冲轮廓')

```

```
ax3.plot( $\phi\_x$ , h_y, c='blue', label='标准脉冲轮廓')
ax3.set_xticks(np.arange(0, 1.01, 0.2))
ax3.set_xlabel('相位', fontsize=14)
ax3.set_ylabel('脉冲轮廓', fontsize=14)
ax3.tick_params(axis='x', labelsz=14)
ax3.tick_params(axis='y', labelsz=14)
ax3.legend(loc='upper center')
plt.savefig('4_3仿真脉冲轮廓和标准脉冲轮廓.pdf',
            pad_inches=None, bbox_inches='tight', dpi=300)
plt.show()

main()
```